

I/O-Efficient Multi-Criteria Shortest Paths Query Processing on Large Graphs

Xinjie Zhou, Kai Huang, Lei Li, Mengxuan Zhang, and Xiaofang Zhou, *Fellow, IEEE*

Abstract—Shortest path computation is a basic operation for many graph-based applications and has been extensively studied. However, most existing works only consider the optimal path of a single criterion but ignore real-world situations involving multiple criteria. This paper investigates a new Multi-Criteria Shortest Paths (MCSPs) problem, aiming to compute the shortest paths of all criteria between a vertex pair. It is significant for real-world applications such as GPS navigation and social network analysis. Nevertheless, the rapid growth of graph size or memory-limited devices poses a memory-constraint challenge, making the adaptation of existing methods extremely time-consuming. To solve the memory-constraint MCSPs problem, we propose a general STOP & SHARE scheme to synchronize the search speeds of all criteria for sharing partition accesses. Two algorithms called *OHP* and *MHP*, adopting the one-hop strategy and partition exhaustive strategy, respectively, are proposed for implementing our scheme. Moreover, we develop two optimized algorithms, *BMHP* and *BMHPS*, to improve query efficiency by combining *MHP* with the bidirectional technique and a novel *in-partition shortcut optimization*. We also investigate *partition-oriented I/O management*. Experimental studies on large real-world graphs demonstrate the effectiveness of the proposed methods over the multi-pass adaptations of the existing methods.

Index Terms—Shortest Paths, Multi-criteria, I/O-efficient, Large Graph

1 INTRODUCTION

SHORTEST path (SP) computation is one of the most fundamental problems on graphs and nurtures a wide range of applications such as GPS navigation, social network analysis, etc. A graph is denoted as $G = (V, E)$, where V is the vertex set and E is the edge set. Each edge $e \in E$ is associated with a single value like distance on road network or a set of values (*i.e.*, multiple criteria such as distance, travel time, greenhouse gas emissions, toll fee, etc.). The shortest path query $Q(s, t)$ aims to retrieve a path with the minimum cost from vertex s to t . Classic shortest path algorithms can be categorized into two types: the *index-free* [17], [24], [29], [40], [58], [85] that are slower to compute but can run on the graph directly without preprocessing, and the *index-based* [3], [13], [22], [38], [42], [43], [54], [60], [64] that are fast to run but take time to construct and maintain the index when network evolves [40], [82], [86], [87]. As the real-world network is naturally dynamic, the *index-free* algorithms that do not need index construction and maintenance are more appealing to industry [40].

However, the aforementioned methods only consider a single criterion of interest, ignoring that real-world networks may involve multiple criteria. On road networks,

they usually only compute the paths with either the minimum distance, travel time [39], [40], or greenhouse gas emissions [66], thus fail to satisfy diverse user preferences. For instance, user A prefers the shortest distance, while user B is sensitive to travel time. Since we are unaware of the user's preference, we have to provide several potential choices for him/her. In fact, DiDi¹ also provides three types of paths (shortest, fastest, and cheapest) for customers. Besides, people interact in different ways in social networks (*e.g.*, retweeting, mentioning, and replying in Twitter [59]), which can be modeled as multiplex networks with multiple relation types [59]. Similarly, different similarities [70] of the web graphs can also be modeled as an attributed multiplex network [33] where each layer represents a kind of similarity measure. The shortest paths of multiple criteria could facilitate the connection strength analysis [23], betweenness computation [23], [68], and social network visualization [20] on these graphs, thus providing a more comprehensive and informative view of the connections/visualizations between two vertices and making it easier to analyze and understand the graph structure. Note that unlike traditional *Multi-Constrained Shortest Path* [34], [44]–[46], [75], [76] which returns a single path considering all criteria, the *Multi-Criteria Shortest Paths (MCSPs) query* reports a set of shortest paths where each path associates with one criterion.

Moreover, as the graph grows rapidly and the number of criteria increases, it is infeasible to load the entire graph into the memory of most servers [21], [90]. For example, the number of Facebook² users has grown to more than 2.91 billion, where each user has about 338 friends on average. Such a massive social network will consume about 7.2 TB to store the graph structural information, let alone

- Xinjie Zhou is with the Hong Kong University of Science and Technology, Hong Kong, China. E-mail: xzhouby@connect.ust.hk
- Kai Huang is with Macau University of Science and Technology. E-Mail: kylehuangk@gmail.com
- Lei Li (✉)^{2,1} and Xiaofang Zhou^{1,2} are with ¹Department of CSE, the Hongkong University of Science and Technology, and ²Thrust of DSA, the Hong Kong University of Science and Technology (Guangzhou). E-Mail: thorli@ust.hk, zxf@cse.ust.hk
- Mengxuan Zhang is with the University of North Texas, US. E-Mail: mengxuan.xzhang@unt.edu
- Corresponding Author: Lei Li

Manuscript received April 19, 2005; revised August 26, 2015.

1. <https://www.didiglobal.com/>

2. <https://about.facebook.com/company-info/>

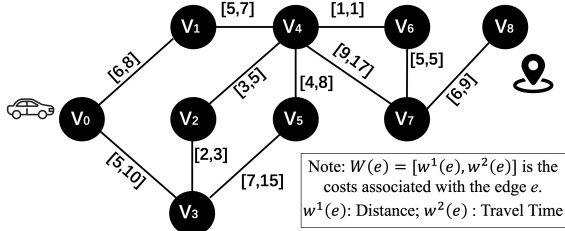


Fig. 1. Example Graph G .

the weights of multiple criteria. Although deploying more servers with larger memory and resorting to a cluster-based system is a solution, it is resource-inefficient as it requires a costly investment in a powerful computing infrastructure to handle big graphs [71]. For example, it requires a 64-node cluster with 1024 processors and 16TB memory to process *single-source shortest path* (SSSP) queries on graphs with tens of billions of edges [81]. Such a large cluster of machines is often beyond the reach of many companies with limited resources [18]. Moreover, using more processors for cluster-based systems may incur a higher communication cost [18] which can even dominate the overall processing time [81]. Therefore, out-of-core algorithms [1], [2], [8], [11], [50], [51], [53], [90] that exploit the external memory are widely investigated to scale up the shortest path computation on a single machine. However, they only consider one criterion and thus fail to share computation among different criteria. For road networks with relatively small sizes, we may have to plan multi-criteria routes on mobile systems with limited memory and low performance as mobile broadband is not available everywhere [4]. For example, online route planning that leverages remote servers to compute the shortest paths and send them back to the mobile device is not available for rural areas where the internet connection is often poor [4]. Hence, the memory-constrained multi-criteria shortest paths query processing plays an important role in practice.

Example 1. The edges in Figure 1 have two criteria: distance and travel time. Given a query $q = (v_0, v_8)$, one may prefer the shortest path $sp^1 = \langle v_0, v_3, v_2, v_4, v_6, v_7, v_8 \rangle$ of 22 km, while another may prefer the fastest path $sp^2 = \langle v_0, v_1, v_4, v_6, v_7, v_8 \rangle$ of 30 min. To meet the diverse preferences, both sp^1 and sp^2 are returned to users. However, it is inefficient to compute them individually, especially when G is a large dynamic graph that can hardly be stored in memory.

Unfortunately, most existing *index-based* methods (e.g., CH [22], PLL [3], H2H [60]) need to load the entire graph into the main memory for index construction, which makes them impossible to maintain the index update under memory-constraint scenarios. Besides, it is inefficient to answer our problem even with the I/O-efficient SSSP algorithm like *EM_Dijk* [51] or general out-of-core graph systems [14], [49], [81], [91] as they fail to share the computation among different criteria. To find the shortest paths of multiple criteria, they have to process each criterion iteratively and individually, resulting in repeated loading of edge information for each criterion. Since the criteria of the same edge are stored together, it would be beneficial to take advantage of this property and share the I/O operations among the path computations.

Motivated by the above observation, in this paper, we investigate I/O-efficient algorithms to process shortest paths queries of multiple criteria simultaneously on large graphs which cannot be entirely held in memory. However, it is challenging to achieve this goal. Firstly, it is hard to track and share the visited vertices for the multiple Dijkstra's traversals since each of them is an individual best-first searching, thus their traversal orders and search spaces are data-dependent. As a result, it hinders information sharing among the shortest path computations of different criteria. Secondly, since the expansion boundary grows rapidly and uniformly as the search expands [6], it reduces the overlapping of the required data of different criteria, thus greatly decreasing the probability of data sharing. Thirdly, since partitions usually organize the data to take advantage of its data locality [50], [51], [79], frequent evictions of under-utilized partitions and massive redundant I/O operations may be involved due to the rapid growth of expansion boundaries and the limited memory. For example, the same partition may be loaded into memory multiple times before being fully used by all criteria because it has to be evicted to make room for other partitions.

Contributions. Our contributions are listed as follows.

- To our best knowledge, we are the first to formally investigate the problem of processing multi-criteria shortest paths queries on massive disk-resident graphs.
- We propose a novel STOP & SHARE scheme and two I/O-efficient algorithms named *OHP* and *MHP* to share computations among multiple criteria by synchronizing their search speeds.
- We develop two optimized algorithms, *BMHP* and *BMHPS*, to further improve query efficiency by integrating bidirectional search with *MHP* and building shortcuts among boundary vertices for each partition, respectively.
- We also propose *partition-based I/O management*, which further improves efficiency by adopting an I/O-efficient compressed sparse row (CSR) index and a combination of the least recently used (LRU) strategy, *toss-immediate* strategy, and *partition shrinking* strategy.
- Extensive evaluations on large real-world graphs demonstrate the effectiveness and efficiency of our methods over the multi-pass³ adaptations of the single-criterion shortest path methods.

2 PRELIMINARY

A graph is represented as $G = (V, E)$ where V and $E \subseteq V \times V$ are the vertex set and the edge set, respectively. Each edge $e \in E$ is associated with a weight vector $W(e) = [w^1(e), \dots, w^k(e), \dots, w^N(e)]$ where $w^k(e)$ is the weight/cost of k^{th} criterion and N is the number of criteria. The size of the graph is $|G| = |V| + N \cdot |E|$. In addition, each v 's neighbors are denoted as $N(v) = \{u : (u, v) \in E\}$ and its degree is $deg(v) = |N(v)|$. Note that we focus our discussions on weighted, undirected graphs while it is easy to extend our techniques to the directed ones. Table 1 lists the key notations and acronyms used in this paper.

3. Note that we refer to methods that iteratively process each criterion as *multi-pass* methods while methods that process all criteria simultaneously as *one-pass* methods in this paper.

TABLE 1
List of key notations.

Notation	Description
N	The number of criteria, $N \geq 2$
G_i	A subgraph of partition i
B and B_i	Boundary vertex set of all partition and partition i
α and μ	Eviction threshold and Memory ratio of I/O management

A *path* $p_{s,t}$ from s to t is a sequence of consecutive vertices $p_{s,t} = \langle s = v_0, v_1, \dots, v_k = t \rangle = \langle e_0, e_1, \dots, e_{k-1} \rangle$. The length of p on the k^{th} criterion is total sum of $w^k(e)$ of $e \in p$, i.e., $len(p_{s,t}^k) = \sum_{e \in p} w^k(e)$. The *shortest path* sp^k of the k^{th} criterion is the path with the minimum path length $len(p^k)$, i.e., $sp^k = \arg \min_{p \in \mathcal{P}} len(p^k)$ where $\mathcal{P}$ is all paths from s to t . The *shortest distance* of $p_{s,t}^k$ on the k^{th} criterion is denoted as $d^k(s, t)$. The *shortest paths of multiple criteria* is a set of shortest paths $SP = \{sp^1, \dots, sp^N\}$.

Definition 1 (Multi-Criteria Shortest Paths (MCSPs) Problem). Given the graph $G = (V, E)$ with N criteria, $MCSPs(s, t)$ finds all paths $SP = \{sp^1, \dots, sp^k, \dots, sp^N\}$, where sp^k is the shortest path from s to t w.r.t. k^{th} criterion.

Example 1 shows $MCSPs(v_0, v_8)$ on a graph with 2 criteria. Next, we introduce some terms for our algorithms.

Definition 2 (Shared Vertex, Shared Partition, and Shared Partition Access). A vertex v is called *Shared Vertex* if its adjacency list are loaded into memory by criterion i and accessed by another criterion j . A partition G_i is called *Shared Partition* if it contains some shared vertices. If shared partition G_i reduces the partition access of criterion j , we say that there is *Shared Partition Access* between criterion i and j .

3 STOP & SHARE: A NOVEL SCHEME

To address the aforementioned limitations, we first propose a novel scheme called STOP & SHARE and its straightforward implementation ONE-HOP. Note that all proposed algorithms under STOP & SHARE scheme in this paper are semi-external memory algorithms that assume the main memory is large enough to hold some constant bits of information per vertex but not the whole graph (i.e., $\delta \cdot |V| \leq M < |G|$) [37], [89], where δ is a constant number and M is the main memory size. In the following, we use M to denote the main memory size, B the disk block size where $1 \ll B \leq M/2$. The disk scan cost is $scan(D) = \Theta(\frac{D}{B})$ and the sorting cost is $sort(D) = \mathcal{O}(\frac{D}{B} \log \frac{M}{B} (\frac{D}{B}))$, where D is the amount of data read/write from/to disk [37].

3.1 Multi-Criteria Graph Partitioning

The essence of I/O-efficient graph processing algorithms is avoiding random I/Os [50], [51], [79]. One effective approach is to divide the graph into several partitions to utilize the data locality (i.e., when we access a vertex v , we read all vertices in v 's partition C because they are "close" to each other and thus have a high possibility to be visited soon) as shown in *MM_BFS* [50] and *EM_Dijk* [51]. Therefore, our STOP & SHARE scheme also partitions the graph by adapting the *deterministic graph clustering* [51] to the multi-criteria scenario. This method obtains the connected components by

minimum spanning tree (MST) and leaves the heavy weights for cutting. In our case, it is more difficult to move to other partitions until the loaded partition is fully utilized.

To handle the multi-criteria scenario, we first generate a representative graph for the original graph by assigning a single weight for each edge according to different edge aggregation strategies (primary (PRI), average (AVE), and weighted average (WAVE)). Then, we compute *MST* of the representative graph and generate an Euler tour over the *MST* by list-ranking and sorting. Lastly, we chop the Euler tour to size-balanced partitions according to the predefined partition size. In particular, PRI picks the edge weight of one primary criterion (the most important criterion in downstream application), while AVE uses the average weight of all criteria. WAVE assigns each criterion a weight according to the prior knowledge (e.g., the query distribution) and attaches the weighted average value to the representative edge. Specifically, the criterion involved in more queries should have a larger weight proportion.

3.2 Stop-and-Share Scheme

The key idea of STOP & SHARE is to simultaneously process multiple criteria by synchronizing their search speeds to share I/O computations. In particular, it maintains multiple priority queues where each stores the vertices in a best-first traversal. We synchronize the traversal speeds of all criteria by synchronizing the hops each priority queue has been processed. Compared to processing iteratively for each criterion, STOP & SHARE can reduce the redundant I/O operations by sharing the partition accesses.

Share Scheme. Let $PQ^1, \dots, PQ^N$ be the priority queues of different criteria and the top elements of them are $top^1, \dots, top^N$. The *share scheme* shares access to partition G_i if at least two criteria need to access it. Furthermore, if G_i has already been loaded into memory by a criterion, the other criteria can share the information of G_i if their top elements are located in G_i , which decreases I/O number.

Stop Scheme. If none of the top elements share the same partition ($top^1, \dots, top^N$ are located in different partitions), the *stop scheme* stops the traversal process of some criteria by one or multiple hop(s) to increase the information sharing probability among all criteria in the subsequent traversals. In other words, the stop scheme aims to maximize shared disk data access by adopting different synchronization strategies. The intuition is that if one partition G_i to be read by criterion k is destined to be accessed by other criteria, pausing the computation on criterion k may lead to subsequent shared access to partition G_i . Compared to iteratively executing the computation for each criterion until finding the shortest path and repeating such a process, applying a synchronization strategy among all criteria may increase the shared probability of partition accesses if the stop scheme is deliberately designed.

The question becomes how and to what extent we can benefit from the *Stop-and-Share* scheme. We try to answer the latter one by investigating the problem of finding the maximum number of shared partition access in multiple priority queues. However, it is very hard to solve this problem because it depends on the vertex traversal orders of all criteria, graph partition algorithm, partition size, and

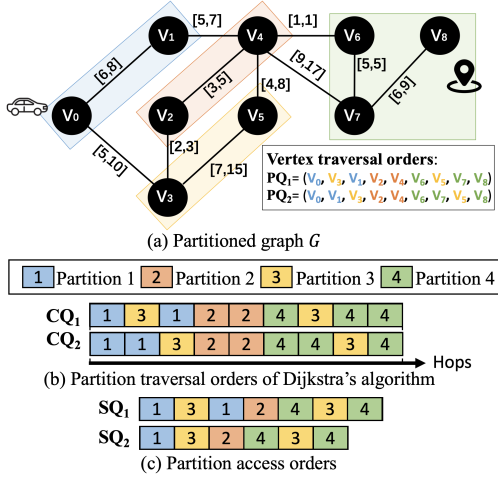


Fig. 2. Example Graph G with Partitions.

available main memory size. In this paper, we try to solve this problem by considering the simple case that the partition size is balanced and the main memory size can only hold one partition. For ease of presentation, we begin with two criteria.

Given a graph G and any two criteria, let PQ_i and PQ_j be their priority queues. Suppose that we have an oracle that knows $PQ_i = \{v_1^i, \dots, v_{M_i}^i\}$ and $PQ_j = \{v_1^j, \dots, v_{M_j}^j\}$ in advance, the corresponding traversal partitions of these vertices are $CQ_i = \{G_1^i, \dots, G_{M_i}^i\}$ and $CQ_j = \{G_1^j, \dots, G_{M_j}^j\}$, where the M_i and M_j are the element number in CQ_i and CQ_j , respectively. We prove the optimal stop scheme with the maximum shared partition accesses can be reduced to Longest Common Sub-Sequence (LCSS) problem if the vertex traversal order is known in Theorem 1.

Theorem 1. Given two queues $CQ_i = \{G_1^i, \dots, G_{M_i}^i\}$ and $CQ_j = \{G_1^j, \dots, G_{M_j}^j\}$, the problem of finding the maximum number of shared partition access between CQ_i and CQ_j can be reduced to the LCSS problem.

Proof. The elements in CQ_i and CQ_j can directly reflect the partition distribution of required data on the scale of hop. CQ_i and CQ_j can be further shrunk into two partition access queues SQ_i and SQ_j by removing repeat sequential elements, e.g., if several sequential elements in CQ_i are the same G_k , we reduce these elements into one G_k in SQ_i . So we can get the maximum number of shared partition access by regarding SQ_i and SQ_j as two strings and solving the LCSS problem between them, which can be solved by dynamic programming with time complexity $\mathcal{O}(|PQ_i||PQ_j|)$. The LCSS for the partition access queues of N criteria can be found in $\mathcal{O}(|V|^{N-1})$ time by an extension of the pairwise algorithm [26]. $\square$

Although the vertex traversal order from source to target has been determined when the query is issued, we have no such oracle to obtain the traversal order before the target is reached. Besides, in most cases, the memory size is large enough to load more than one partition which makes the problem much more difficult. Therefore, we have to resort to heuristics to implement the STOP & SHARE scheme.

Example 2. Reconsider the Example 1, G is divided into four partitions as shown in Fig. 2 (a). The vertex traversal orders

Algorithm 1: OHP Algorithm

Input: $G = (V, E)$ with N criteria, query $MCSPs(s, t)$
Output: Shortest paths of all criteria $SP = \{sp^1, \dots, sp^N\}$

- 1 Allocate a buffer Buf_G for storing G ; $C \leftarrow \phi$;
- 2 **foreach** criterion $i \in [1, N]$ **do**
- 3 Initialize an external min-priority queue PQ_i with $\langle s, 0 \rangle$;
- 4 $C \leftarrow C \cup i$; $dist_i[s] \leftarrow 0$; $prev_i[s] \leftarrow s$;
- 5 **while** $\exists i \in [1, N]$ s.t. PQ_i is not empty **do**
- 6 $C_r \leftarrow C$;
- 7 **while** C_r is not empty **do**
- 8 $k \leftarrow \text{GETFIRSTCRI}(C_r)$; $\langle u_k, cost_k \rangle \leftarrow \text{EXTRACTMIN}(PQ_k)$;
- 9 **if** $u_k \neq t$ **then**
- 10 **if** u_k is not in Buf_G **then**
- 11 $\text{READPARTITION}(v_pt[u_k], false)$;
- 12 $\text{RELAXATION}(u_k)$;
- 13 $\text{STATUSUPDATE}(u_k, true)$; $\triangleright$ For I/O management
- 14 **else**
- 15 $min_cost[k] \leftarrow cost_k$; $PQ_k \leftarrow \phi$; $C \leftarrow C \setminus k$;
- 16 $C_r \leftarrow C_r \setminus k$;
- 17 $SP \leftarrow \text{GETSHORTESTPATHS}(prev)$;
- 18 **Function** $\text{RELAXATION}(u_k)$:
- 19 **foreach** $v \in N(u_k)$ **do**
- 20 **if** $closed_k[v] = false$ **then**
- 21 $tmp \leftarrow cost_k + w_k(u_k, v)$;
- 22 **if** $tmp < dist_k[v]$ **then**
- 23 $dist_k[v] \leftarrow tmp$; $prev_k[v] \leftarrow u_k$;
- 24 $PQ_k.\text{UPDATE}(\langle v, dist_k[v] \rangle)$;
- 25 $closed_k[v] \leftarrow true$;

(i.e., PQ_1 and PQ_2) and partition traversal orders (i.e., CQ_1 and CQ_2) are shown in Fig. 2 (a) and (b). The partition access queues SQ_1 and SQ_2 are shown in Fig. 2 (c). Assume that the main memory is large enough to only load one partition, then the maximum shared partition is the same as the LCSS of SQ_1 and SQ_2 , which is $\langle G_1, G_3, G_2, G_4, G_3, G_4 \rangle$. Therefore, the maximal shared partition access number is 6 and the minimal number of partition access is $7 + 6 - 6 = 7$.

3.3 The Basic STOP & SHARE Algorithm: OHP

We first propose the basic algorithm named *One-Hop (OHP)* for the STOP & SHARE scheme. The idea of OHP is to execute the traversal process of all criteria for *one* hop in each round (i.e., *One-Hop Strategy*) to synchronize their search speeds, thus increasing the shared probability.

One-Hop Strategy (OHS) aims to alternatively perform multiple Dijkstra's traversal processes hop by hop to seek sharing access to partitions among all criteria. Specifically, we execute each criterion that has not reached the target vertex by one hop in each round and read the corresponding partition immediately as long as the top element is not in the main memory. The intuition behind this strategy is that the one-hop strategy can synchronize the expansion speed of all criteria, thereby confining their search space into the localized region around the expansion boundary.

The pseudo-code of OHP is outlined in Algorithm 1. OHP first allocates a buffer Buf_G to store G , where $|Buf_G| < M$. Note that a deliberately-designed I/O management strategy of Buf_G will be introduced in Section 5. For each criterion $i \in [1, N]$, it initializes an external-memory min-priority queue PQ_i provided by STXXL library [16] with $\langle s, 0 \rangle$ (source s and distance 0), and records

the vector of distances to source vertex ($dist_i$) and the predecessors ($prev_i$) (Lines 3 to 4). Besides, all criteria are added to the unfinished-criteria set $\mathcal{C}$. If there exists at least one non-empty priority queue, the remaining criteria set $\mathcal{C}_r$ will be initiated with $\mathcal{C}$ in each round. Lines 8 to 16 demonstrate the core idea of *One-Hop Strategy* where each criterion will be processed alternatively, where GETFIRSTCRI is the function of retrieving a criterion from set $\mathcal{C}_r$. The process repeats until all priority queues are empty or destination t is reached. The procedure of vertex relaxation is shown in Lines 19 to 25, where $closed_k$ is the vector of boolean value recording whether a vertex has been visited. Finally, all shortest paths SP are derived from predecessor arrays by tracing backward from target to source (Line 17).

Example 3. Reconsider the Example 2, criterion 1 (Cri_1) and criterion 2 (Cri_2) will be executed hop by hop alternatively. Therefore, if only one partition can exist in memory, G_1 will first be loaded by Cri_1 and shared by Cri_2 in the 1st round. In the 2nd round, the loading of G_3 by criterion 1 leads to the eviction of G_1 in memory, resulting in the extra reload of G_1 by Cri_2 . Luckily, the reloaded G_1 can be utilized by Cri_1 in the 3rd round. The above process will carry on until the shortest paths of all criteria are found. The final partition access order is $\langle G_1, G_3, G_1, G_3, G_2, G_4, G_3, G_4, G_3, G_4 \rangle$, resulting in 6 shared partition accesses, 3 extra partition accesses and $7 + 6 - (6 - 3) = 10$ partition accesses from disk.

Complexity Analysis. The space complexity is $\mathcal{O}(M)$. The I/O complexity of N STXXL priority queues [16] is $\mathcal{O}(\frac{N}{B} \log_{\frac{M}{B}}(\frac{|E|}{B}))$. Since the I/O complexity for loading all partition files for one criterion is $\mathcal{O}(\frac{|V|}{|C|} + \frac{|E|}{B})$ where $|C|$ is the average partition size [53], the amortized I/O complexity for each vertex is $\mathcal{O}(\frac{1}{|C|} + \frac{N|E|}{|V||B|})$ when N criteria are involved. Therefore, the overall I/O complexity of OHP is $\mathcal{O}(\frac{N}{B} \log_{\frac{M}{B}}(\frac{|E|}{B}) + \sum_{i=1}^N (|V_i| - |V_i^{OHP}|)(\frac{1}{|C|} + \frac{N|E|}{|V||B|}))$, where V_i and V_i^{OHP} are the visited vertex set and shared vertex set of criterion i , respectively.

3.4 Multi-Hop Algorithm: MHP

Observe that with the expansion of Dijkstra's traversal process, the search frontier (i.e., the vertices waiting for traversal in the priority queue) enlarges exponentially in small-world graphs, while it becomes geographically large in road networks [6]. Moreover, the search frontier usually expands uniformly. As a result, there is a high probability that when we expand the search space in one partition G_i , Dijkstra's algorithm may expand the search space in all other partitions along the search front before coming back to G_i [6]. In this circumstance, G_i may have to be evicted from memory, resulting in expansive reloads. Hence, during the search expansion of one criterion, it is beneficial to thoroughly exploit existing in-memory information before moving on to another criterion. Inspired by these observations, we propose an improved Multi-Hop (MHP) algorithm to implement the stop-and-share scheme based on a novel *Partitions-Exhaustive Strategy* (PES). PES aims to share the in-memory information of accessible vertices as much as possible during the synchronized searches of all criteria. Specifically, PES continues to process each criterion

Algorithm 2: MHP algorithm

Input: $G = (V, E)$ with N criteria, query $MCSPs(s, t)$
Output: Shortest paths of all criteria $SP = \{sp^1, \dots, sp^N\}$

- 1 **INITIALIZATION;** /*Lines 1 to 4, Algorithm 1*/
- 2 **foreach** criterion $i \in [1, N]$ **do**
- 3 $H[i] \leftarrow 0$; $cri_p[i] \leftarrow v_pt[s]$;
- 4 initialize a min-priority queue HQ with $\langle i, 0 \rangle$;
- 5 **while** $\exists i \in [1, N]$ s.t. PQ_i is not empty **do**
- 6 $\mathcal{C}_r \leftarrow \mathcal{C}$;
- 7 $READPARTITION(cri_p[GETTOPKEY(HQ)], false)$;
- 8 **while** $\mathcal{C}_r$ is not empty **do**
- 9 $k \leftarrow GETFIRSTCRI(\mathcal{C}_r)$; $u_k \leftarrow TOP(PQ_k)$;
- 10 **while** u_k is in Buf_G **do**
- 11 $\langle u_k, cost_k \rangle \leftarrow EXTRACTMIN(PQ_k)$;
- 12 **if** $u_k \neq t$ **then**
- 13 $RELAXATION(u_k)$; $STATUSUPDATE(u_k, true)$;
- 14 $H[k] \leftarrow H[k] + 1$; $u_k \leftarrow TOP(PQ_k)$;
- 15 **else**
- 16 $min_cost[k] \leftarrow cost_k$; $PQ_k \leftarrow \phi$; $\mathcal{C} \leftarrow \mathcal{C} \setminus k$;
- 17 $H[k] \leftarrow \infty$; **break**;
- 18 $\mathcal{C}_r \leftarrow \mathcal{C}_r \setminus k$; $cri_p[k] \leftarrow v_pt[u_k]$;
- 19 $HQ.UPDATE(\langle k, H[k] \rangle)$;
- 20 **end while**
- 21 $SP \leftarrow GETSHORTESTPATHS(prev)$;

(may execute each criterion multiple hops in each round) until the top element in its priority queue is not accessible in memory or the target vertex t is reached, then moves on to other criteria.

The pseudo-code of MHP is outlined in Algorithm 2. In each round, it first loads a new partition according to the *Minimum-Hops-First Strategy*, i.e., it only reads the partition required by the criterion with minimal hops in each round. To implement this strategy, it maintains two arrays $H[i]$ and $cri_p[i]$ recording the number of processed hops and the partitions to read for all criteria, and a minimum priority queue containing the number of processed hops of all criteria (Lines 1 to 4). After partition loading, all criteria are processed according to *Partitions-Exhaustive Strategy* (Lines 7 to 16). The above process repeats until all priority queues are empty, then the shortest paths can be obtained according to the predecessor arrays.

Example 4. Reconsider the Example 2, MHP first loads G_1 in 1st round, leading to one shared partition access, one hop processing (i.e., v_0) in CQ_1 and two hops processing (i.e., v_0, v_1) in CQ_2 . In the 2nd round, G_3 is loaded by Cri_1 after G_1 is evicted, resulting in one shared partition access and the processing of v_3 in both CQ_1 and CQ_2 , and so on. The final partition access order is $\langle G_1, G_3, G_1, G_2, G_4, G_3, G_4 \rangle$, with 6 shared partition accesses and $7 + 6 - 6 = 7$ partition accesses from disk, which is the same as the optimal result.

Complexity Analysis. Similar to OHP, the space complexity of MHP is $\mathcal{O}(M)$ while the I/O complexity is $\mathcal{O}(\frac{N}{B} \log_{\frac{M}{B}}(\frac{|E|}{B}) + \sum_{i=1}^N (|V_i| - |V_i^{MHP}|)(\frac{1}{|C|} + \frac{N|E|}{|V||B|}))$, where V_i^{MHP} is the shared vertex set of criterion i .

4 OPTIMIZATIONS OF STOP & SHARE SCHEME

The key idea of *Stop-and-Share* scheme is to synchronize the search expansions of all criteria such that they can share common partition accesses to reduce redundant I/O operations. However, the search frontiers of all criteria will enlarge

dramatically with the expansions of Dijkstra's traversal. Worse yet, for small-world graphs (e.g., social network and web graphs) which have a high average degree and uneven degree distribution, it is more likely to make the unidirectional search algorithms such as *OHP* and *MHP* suffer from exponentially enlarging search fronts. As a result, massive evictions of underutilized partitions will be inevitable under limited memory and thus involve numerous expensive I/O operations. We call this reaching the I/O "*Tipping Point*" of Dijkstra's search expansion. In this section, we propose two optimization techniques to alleviate the effect of the "*Tipping Point*", thus improving the query efficiency of the *Stop-and-Share* scheme.

4.1 Bidirectional Multi-Hop Algorithm: BMHP

Bidirectional technique [58] is a general and effective approach to speed up the query performance of Dijkstra's algorithm [24]. Its key idea is to search from both the source and the target simultaneously such that Dijkstra's search space is narrowed down to two smaller circles centered at source vertex s and target vertex t [24]. Therefore, we first further propose the bidirectional version of *MHP*, called *BMHP*, which adopts *Bidirectional Partitions-Exhaustive Strategy (BPES)* to effectively prune the search space and minimize the size of the search frontier. *BPES* also targets making full use of the accessible vertices in memory to share the partition access. In particular, it continues to execute the forward search and reverse search of one criterion for multiple hops as long as the top element of the priority queue is accessible.

It is worth noting that different from bidirectional Dijkstra's algorithm [58] that performs only one hop for the forward search and reverse search in each round, the processed hops of *BPES* might be different in the forward search and reverse search of the same criterion as its goal is to exploit existing in-memory information. As such, when the destination partition is loaded into memory, the forward/reverse search could directly reach the destination vertex as per the *BMHP* algorithm. To address this situation, we propose Proposition 1 as termination conditions for *BMHP*.

Proposition 1. *There are two Termination Conditions (TC) in BMHP as follows. For any criterion i , (1) TC1: if the cost of the top elements of forward search plus the cost of the top elements of reverse search is larger than the current minimal cost, or (2) TC2: if either the forward search or the reverse search reaches the destination (i.e., t for forward search, s for reverse search), then we can terminate the processing of BMHP w.r.t. criterion i and get corresponding shortest path.*

Theorem 2. *The Termination Conditions in Proposition 1 are correct for BiMULTI-HOP algorithm.*

Proof. There are two cases for the locations of source vertex s and target vertex t : (1) s and t locate in different partitions, or (2) s and t locate in the same partition. For case (1), both forward search and reverse search have a chance to be executed. Nonetheless, the expansion speed may differ among them. Therefore, when they meet in the middle, the update of minimum cost and TC1 can ensure the shortest path is found according to [58]. For case (2), after the loading of G_k , the forward search may have reached the target

Algorithm 3: BMHP algorithm

Input: $G = (V, E)$ with N criteria, query $MCSPs(s, t)$
Output: Shortest paths of all criteria $SP = \{sp^1, \dots, sp^N\}$

- 1 **INITIALIZATION** of Forward and Reverse Search; $\triangleright$ Similar to Lines 1 to 4 of Algorithm 2. Omit here.
- 2 **while** $\exists i \in [1, N]$ s.t. PQ_i is not empty **do**
- 3 $C_r \leftarrow C_i$;
- 4 $BiREADPARTITION(cri_p[TOP(HQ)], cri_p^r[TOP(HQ^r)]);$
- 5 **while** C_r is not empty **do**
- 6 $k \leftarrow GETFIRSTCRI(C_r); u_k \leftarrow TOP(PQ_k); u_k^r \leftarrow TOP(PQ_k^r);$
- 7 **if** $TOP(PQ_k) + TOP(PQ_k^r) \geq \min_cost[k]$ **then**
- 8 $PQ_k \leftarrow \phi; PQ_k^r \leftarrow \phi; C \leftarrow C \setminus k;$
- 9 $H[k] \leftarrow \infty; H^r[k] \leftarrow \infty;$
- 10 **else**
- 11 /* Forward Search */
- 12 **while** u_k is in Buf_G and $closed_k^r[u_k] = false$ **do**
- 13 $\langle u_k, cost_k \rangle \leftarrow EXTRACTMIN(PQ_k);$
- 14 $H[k] \leftarrow H[k] + 1;$
- 15 **if** $u_k = t$ **then**
- 16 $\min_cost[k] = cost_k[u_k]; PQ_k \leftarrow \phi;$
- 17 $PQ_k^r \leftarrow \phi; C \leftarrow C \setminus k; H[k] \leftarrow \infty;$
- 18 $H^r[k] \leftarrow \infty; \mathbf{break};$
- 19 **foreach** $v \in N(u_k)$ **do**
- 20 **if** $closed_k[v] = false$ **then**
- 21 $ttmp \leftarrow cost_k + w_k(u_k, v);$
- 22 **if** $ttmp < dist_k[v]$ **then**
- 23 $dist_k[v] \leftarrow ttmp; prev_k[v] \leftarrow u_k;$
- 24 $PQ_k.UPDATE((v, dist_k[v]));$
- 25 **if** $cost_k[v] + ttmp < \min_cost[k]$ **and** $closed_k^r[v] = true$ **then**
- 26 $\min_cost[k] = cost_k^r[v] + ttmp;$
- 27 $closed_k[v] \leftarrow true; STATUSUPDATE(u_k, true);$
- 28 $u_k \leftarrow TOP(PQ_k);$
- 29 /* Reverse Search. Similar to Forward Search. Omit here */
- 30 $C_r \leftarrow C_r \setminus k; cri_p[k] \leftarrow v_p[u_k];$
- 31 $HQ.UPDATE((k, H[k])); cri_p^r[k] \leftarrow v_p[u_k^r];$
- 32 $HQ^r.UPDATE((k, H^r[k]));$
- 33 $SP \leftarrow GETSHORTESTPATHS(prev, prev^r);$

vertex before jumping into reverse search. In this case, TC2 can make sure the shortest path has been found as long as the destination is reached according to [17]. $\square$

BMHP is outlined in Algorithm 3. In each round, we first load partitions for the forward search and reverse search according to *Minimum-Hops-First Strategy* to synchronize the search fronts of all criteria. Next, for each criterion, we check whether the shortest path has been found according to the TC1. If not, we keep doing forward search and reverse search according to *BPES* (Lines 11 to 26). This process repeats until the searches of all criteria are terminated.

Example 5. *Reconsider the Example 2. According to BiMULTI-HOP algorithm, the Dijkstra's traversal processes of Cri_1 and Cri_2 are performed from v_0 and v_8 simultaneously. Therefore, G_1 is first loaded in memory, followed by G_4 in the 1st round. Notice that we only need to read G_3 once in the 2nd round which contributes to 4 vertex processing (i.e., v_3 and v_5 in both Cri_1 and Cri_2) in both forward search and reverse search. The final partition access order is $(G_1, G_4, G_3, G_1, G_4, G_2)$, resulting in 7 shared partition accesses and $7 + 6 - 7 = 6$ partition accesses from disk, which is also the optimal solution for bidirectional search.*

Complexity Analysis. The space complexity of *BMHP*

is $\mathcal{O}(M)$ while the I/O complexity is $\mathcal{O}(\frac{N}{B} \log \frac{M}{B} (\frac{|E|}{B}) + \sum_{i=1}^N (|V_i| - |V_i^{\text{BMHP}}|)(\frac{1}{|C|} + \frac{N|E|}{|V||B|}))$, where V_i^{BMHP} is the shared vertex set of criterion i .

4.2 In-Partition Shortcut Optimization

Although *BMHP* has dramatically reduced the I/O operations by pruning the search space, we still have to go through a lot of unnecessary information from visited partitions. A straightforward to further speed up the query processing is to build shortcuts among all boundary vertices to avoid traversal within partitions. However, building shortcuts among all boundary vertices is infeasible due to the $B \times B$ shortcut size and B Dijkstra's traversals on the whole graph. Moreover, such a naive strategy suffers from time-consuming shortcut maintenance when the graph evolves as we have to reconstruct all shortcuts.

To tackle this challenge, we propose a novel *In-Partition Shortcut Strategy (IPS)* to enhance the efficiency of our *Stop&Share* scheme. The key idea of *IPS* is to build minimized shortcuts among boundary vertices of the same partition to preserve the local distances among these boundary vertices. These shortcuts help to reduce computational overhead by bypassing internal traversal within a partition during graph traversals. Furthermore, we can replace the original partition with the sparser shortcut graph to reduce I/O operations. A straightforward implementation of *IPS* is to build full-clique $B_i \times B_i$ shortcuts among the boundary vertices B_i for each partition G_i . However, the full-clique shortcut may fail to reduce the partition size if the original partition G_i is sparse. To solve this problem, we propose *pruned Dijkstra* which adopts two pruning conditions to minimize the in-partition shortcut size and accelerate shortcut construction. We start by considering the connectivity of boundary vertices and classify them into two types:

Definition 3 (Half/Full-Connected Boundary Vertex). In G_i , $b_p \in B_i$ is a half-connected boundary vertex if $\exists u \in N(b_p), u \in G_i, u \notin B_i$ and we denote them as B_i^H ; otherwise, b_p is a full-connected boundary vertex and denoted as B_i^F .

Intuitively, $\forall b_p \in B_i^F$, the first edge on its shortest path to other boundary vertices in B_i only leads to another boundary vertex. Thus the edges between b_p and its neighbors are sufficient to preserve b_p 's connections to other boundary vertices $B_i \setminus b_p$. Based on this observation, we propose *minimized shortcut construction strategy*.

Minimized Shortcut Construction. For each partition G_i , since $|G_i| < M$, we can load it into memory for shortcut construction. In particular, we conduct the *pruned Dijkstra*'s search from each boundary vertex $b_p \in B_i$, adding the shortcuts between b_p and other boundary vertices $b_q \in B_i$ with the following pruning conditions. Pruning Condition 1: For $b_p \in B_i^F$, we only check the edge between b_p and its in-partition neighbor $b_q \in N(b_p) \cap B_i$ and prune the edge if there is another boundary vertex $b_t \in B_i$ lies in the shortest path from b_p to b_q .

Pruning Condition 2: For $b_p \in B_i^H$, we prune the shortcut between b_p and boundary vertex $b_q \in B_i^H$ if there is another $b_t \in B_i$ lies in the shortest path from b_p to b_q .

It is worth noting that the shortcut constructions of each partition are independent, and the pruned Dijkstra's searches for all boundary vertices are also independent. Thus, we could leverage the multi-thread parallelism to accelerate the shortcut construction. We then shrink the original partition G_i into a sparser subgraph $G_i^* = \{V_i^*, E_i^*\}$, where $V_i^* = B_i$, $E_i^* \subseteq \{B_i \times B_i\}$ contains the minimized shortcuts among boundary vertices as per Lemma 1. Then an overlay graph $\tilde{G}$ consisting $\{G_i^*\}$ and cross-partition edges is constructed for *MCSPs* query processing.

Lemma 1. Given a partition G_i , G_i^* produced by pruning conditions 1 and 2 has minimized shortcuts among the boundary vertices in B_i to preserve the local distance of G_i .

Proof. We first prove that E_i^* of G_i^* is sufficient for computing the shortest distance between any two boundary vertices (e.g., $b_p, b_q \in B_i$). There are two cases. Case 1: no intermediate boundary vertex $b_t \in B_i$ lies in the shortest path between b_p and b_q . For this case, the shortcut between them must be added to E_i^* . Case 2: $b_t \in B_i$ lies in the shortest path between b_p and b_q . We take the concise form of $sp(b_p, b_q)$ by extracting only the boundary vertices as $p_c = \langle b_p = b_0, b_1, \dots, b_n = b_q \rangle$ ($b_i \in B, 0 \leq i \leq n$). Since the shortcut $e(b_i, b_{i+1}), 0 \leq i \leq n-1$ all exist in E_i^* according to Case 1, the $d(b_p, b_q)$ could be computed by E_i^* . Next, we prove that removing any edges in E_i^* will violate the local distance preservation of G_i . As per the pruning conditions, for edge $e(b_p, b_q) \in E_i^*$, there must be no $b_t \in B_i$ lies in the shortest path between b_p and b_q . Therefore, the removal of $e(b_p, b_q)$ violates the local distance preservation of $d(b_p, b_q)$. Hence, E_i^* has minimized shortcuts to preserve the local distance of G_i . $\square$

Query Processing. We call the algorithm combining *BMHP* with *IPS* as *BMHPS* and introduce its query processing as follows. Given a query *MCSPs*(s, t), *BMHPS* only need the information of overlay graph $\tilde{G}$ and the original partitions s, t belong to for query processing. In particular, there are two cases for *MCSPs*(s, t).

Case 1: s and t are in the same partition ($s, t \in G_i$). For the subcase that $s, t \in B$, we directly conduct *BMHP*'s search on the overlay graph $\tilde{G}$ until all criteria terminate. Otherwise, we first load the original subgraph G_i and compute the local distances between s (and t) and $b_p \in B_i$, and $d_{G_i}(s, t)$. These values are pushed into corresponding priority queues of *BMHP* as the initial seeds. Then we conduct the *BMHP*'s search on the overlay graph $\tilde{G}$ until the searches of all criteria terminate.

Case 2: s and t are in different partitions ($s \in G_i, t \in G_j, G_i \neq G_j$). For the subcase that $s, t \in B$, we directly conduct *BMHP*'s search on the overlay graph $\tilde{G}$ until all criteria terminate. Otherwise, we first load the original subgraph G_i and G_j into memory. Subsequently, we compute the local distances between s and $b_p \in B_i$ and the local distances between t and $b_q \in B_j$. These local distances are then inserted into the respective priority queues of *BMHP*. Finally, we perform the *BMHP*'s search on the overlay graph $\tilde{G}$ until all criteria terminate.

Green line: shortest path within partition. Yellow line: short path path traverses out of partition

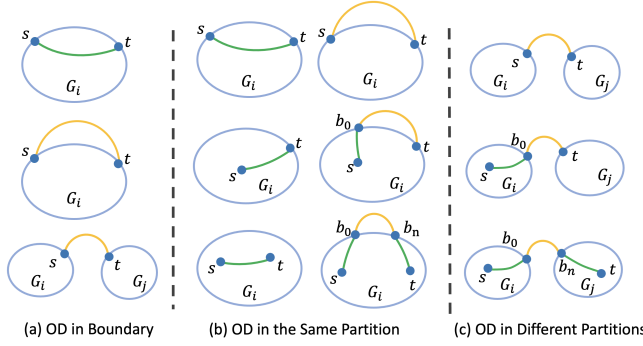


Fig. 3. Illustration of OD Distribution for BMHPS.

Theorem 3. $\forall s, t \in B, d_G(s, t) = d_{\tilde{G}}(s, t)$.

Proof. We divide all the scenarios into three cases, as shown in Figure 3-(a), and prove them in the following:

Case 1: s and t belong to the same G_i and $p_G(s, t)$ only passes through the interior of G_i . Then $d_G(s, t) = d_{G_i}(s, t)$ and $(s, t) \in \tilde{G}$. Since $\tilde{w}(s, t) = d_{G_i}(s, t)$ was used in $\tilde{G}$'s construction, it is obvious that $d_G(s, t) = d_{\tilde{G}}(s, t)$;

Case 2: $s \in G_i$ and $t \in G_i$ but $p_G(s, t)$ goes outside;

Case 3: $s \in G_i$ and $t \in G_j$ from different partitions.

For Cases 2 and 3, we take the concise form of $p_G(s, t)$ by extracting only the boundary vertices as $p_c = \langle s, b_0, b_1, \dots, b_n, t \rangle$ ($b_i \in B, 0 \leq i \leq n$). For two adjacent vertices $b_i, b_j \in p_c$, 1) if $(b_i, b_j) \in E$, then it is the intra-edge with $(b_i, b_j) \in \tilde{G}$ and its correctness is the same as the sub-case 1; 2) if $(b_i, b_j) \notin E$, then it is the inter-edge with $(b_i, b_j) \in \tilde{G}$ and it is naturally correct. Therefore, the shortest distance can be correctly calculated by accumulating only the edge weights on $\tilde{G}$. $\square$

Theorem 4. The BMHPS algorithm can correctly answer the MCSPs queries.

Proof. We divide all the scenarios into two cases, as shown in Figure 3-(b) and (c), and prove them in the following:

Case 1: $s, t \in G_i$. There are three subcases. For subcase $s, t \in B_i$, its correctness has been proved in Theorem 3. For subcase $s \notin B_i, t \in B_i$ (or versa vise), if $p_G(s, t)$ does not go out of G_i , the search on G_i is sufficient for computing it; otherwise, the search on $\tilde{G}$ is also needed for computing $d_G(b_0, t)$, where b_0 lies in $p_G(s, t)$. For subcase $s \notin B_i, t \notin B_i$, we take the concise form of $p_G(s, t)$ by extracting only the boundary vertices as $p_c = \langle s, b_0, \dots, b_n, t \rangle$ ($b_i \in B, 0 \leq i \leq n$), thus $d_G(s, b_0)$ and $d_G(t, b_n)$ are computed by searches on G_i while $d_G(b_0, b_n)$ can be computed by search on $\tilde{G}$.

Case 2: $s \in G_i, t \in G_j, G_i \neq G_j$. There are three subcases. For subcase $s, t \in B$, its correctness has been proved in Theorem 3. For subcase $s \notin B_i, t \in B_j$ (or versa vise), $d_G(b_0, t)$ is obtained from search on $\tilde{G}$ while $d_G(s, b_0)$ is computed by search on G_i . For subcase $s \notin B_i, t \notin B_j$, searches on G_i, G_j , and $\tilde{G}$ are sufficient for computing $d_G(s, b_0)$, $d_G(t, b_n)$, and $d_G(b_0, b_n)$. Therefore, BMHPS can correctly answer the MCSPs queries. $\square$

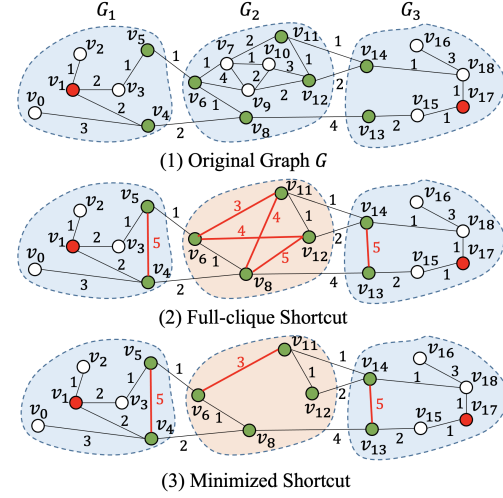


Fig. 4. Illustration of In-partition Shortcut Optimization.

Example 6. Figure 4-(1) shows an original graph G consisting of three partitions $\{G_1, G_2, G_3\}$, where the boundary vertices are painted green. As shown in Figure 4-(2), the naive implementation of IPS simply constructs the shortcuts among boundary vertices of each partition, resulting in six new shortcuts (red lines). Our minimized shortcut construction strategy can further reduce the number of shortcuts (only three new shortcuts), producing the minimized shortcuts. Given $MCSPs(v_1, v_{17})$, BMHPS needs to load G_1, G_3 , and G_2^* . Since G_2^* is much smaller than G_2 , the I/O operations are reduced.

Complexity Analysis. The space complexity of BMHPS is $\mathcal{O}(M)$ while the I/O complexity is $\mathcal{O}(\frac{N}{B} \log_{\frac{M}{B}}(\frac{|\tilde{E}|}{B}) + \sum_{i=1}^N (|\tilde{V}_i| - |\tilde{V}_i^{BMHPS}|)(\frac{1}{|C|} + \frac{N|\tilde{E}|}{|\tilde{V}||B|}))$, where $\tilde{V}_i^{BMHPS}$ is the shared vertex set of criterion i on the overlay graph $\tilde{G}$.

4.3 Discussions

OHP and MHP utilize different search synchronization strategies to implement Stop-and-Share scheme so as to reduce redundant I/O operations. In particular, OHP adopts One-Hop Strategy to perform all criteria alternatively for one hop. While MHP adopt Partitions-Exhaustive Strategy to process some criteria for multiple hops to make full use of the accessible vertices in memory. Since MHP tries to maximize information sharing at the “current moment” by taking advantage of the available information in time, unnecessary reloading of partitions could be eliminated. Thus it is more efficient than OHP in general.

To further improve the efficiency of MHP, we propose two optimized MHP: BMHP combines MHP with the bidirectional technique to prune search space and reduce the search frontier size, while BMHPS further improve BMHP by in-partition shortcut optimization. The in-partition shortcuts of BMHPS could be considered a light index. Fortunately, since the partition size $|G_i|$ is much smaller than the available memory M , we can load multiple partitions into memory and conduct the pruned Dijkstra's searches in parallel to accelerate the shortcut construction and maintenance. Furthermore, this lightweight index can be updated rapidly. In particular, given an updated edge $e(a, b)$, if $e(a, b)$ is an inter-partition edge ($a \in B_i, b \in B_j$), we only need

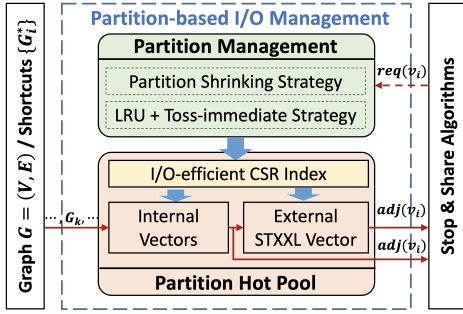


Fig. 5. Overview of I/O Management.

to update the corresponding edge weight in G_i^* and G_j^* . For other cases, we can load the updated partition G_i and G_j into memory and conduct *pruned Dijkstra's* searches in parallel to update the shortcuts in G_i^* and G_j^* . Therefore, *BMHPS* can be seen as an “almost index-free” method and it could be applied in most dynamic environments.

5 PARTITION-ORIENTED I/O MANAGEMENT

One straightforward I/O management policy for handling the partitions in the STOP & SHARE scheme is to utilize traditional page replacement policies such as *LRU* (*Least Recently Used*) [12], by regarding each partition as one page. However, it is I/O inefficient when massive partition reloads are involved. For example, the whole partition has to be reloaded even if we only need to read one vertex and its adjacency list.

A better approach is to leverage the hot pool management proposed in *EM_Dijk*, which classifies edges according to their weight categories and manages them in category-oriented hot pools (each edge category adopts a hot pool that uses a multimap as the internal pool and an *LRU* policy-based external vector as the external pool). However, this approach suffers from several drawbacks: 1) it is expensive to locate the adjacency list of a vertex by scanning the external vector (denoted as **D1**); 2) the internal multimap may be inefficient when the graph has many high-degree vertices because the hash overflows (**D2**); 3) the external vector cache may be too small to efficiently handle the “*Tipping Point*” problem when massive underutilized data resides in external pools (**D3**); 4) good performance relies on the prior knowledge of the edge weight distribution (as we can allocate more memory for the category with more edges) which is typically unknown or unavailable in real-world scenarios. As such, the cache allocation for less frequent edge categories may result in wasted resources. (**D4**). As I/O management is a key part of our methods, we aim to solve these drawbacks and develop efficient partition-based I/O management strategies to improve query efficiency.

5.1 Overview of I/O Management

Observe that many I/O efficient algorithms [1], [51] achieve high efficiency by partitioning graphs into clusters and managing them with hot pools. Therefore, we also propose a partition-oriented graph hot pool to manage the loaded partitions, which naturally solves the drawback **D4** as we allocate memory for partitions instead of edge categories.

The overview of the partition-based I/O management is shown in Figure 5. Specifically, there are two parts in the graph hot pool: *Partition Management* and *Partition Hot Pool*. For *partition management*, we propose a novel combination of *LRU* Strategy, *Toss-immediate* Strategy [67], and *Partition Shrinking Strategy* (denoted as *LTP*) to efficiently manage the partitions (see Section 5.2). The *graph hot pool* leverages a set of internal vectors instead of a multimap to store the partition in memory (each internal vector represents a partition), thereby tackling the drawback **D2**. Besides, we store the unused data of the evicted partitions in an external STXXL vector for future reference. This design significantly enhances the efficiency of partition reloading as the STXXL vector can manage these data in smaller blocks. However, retrieving data in vectors is difficult as the vertex IDs in a partition may not be sequential. To tackle this challenge, we propose a novel *I/O-efficient Compressed Sparse Row (IOCSR) index* to efficiently retrieve graph data in the partition hot pool. Note that *IOCSR index* may consume some constant space for each vertex. Nevertheless, it can greatly improve the efficiency by discarding computationally expensive scanning of hot pools used in *EM_Dijk* and thus solves **D1**. It is worth noting that the STXXL vector also has a cache to efficiently retrieve blocks, and it is easy to reach the “*Tipping Point*” in small-world graphs when the STXXL vector cache is too small. Hence, we introduce a parameter named *memory ratio* μ for the STXXL vector. This parameter allows flexible allocation of memory for STXXL vector, thus alleviating **D3**.

Example 7. Reconsider Figure 5. Assuming that a vertex accessing request $req(v_i)$ from the STOP & SHARE algorithms, the partition management engine first judges whether v_i is in the data pool. If not, the partition G_k where v_i locates is read from the disk and stored in the corresponding internal vector. If there is no spare memory, some in-memory partitions are evicted by either directly discarding from memory or storing into external vector according to the eviction threshold. Finally, v_i and its adjacency list are retrieved from the data pool (either internal vector or external STXXL vector) according to its vertex index.

5.2 I/O-efficient CSR Index and LTP Strategy

Compressed sparse row (CSR) [74], [80] is an efficient data format to retrieve sparse graph data and matrix [80]. It uses a column array and an index array to store all adjacent vertex information and the starting offset in the column array for each vertex, respectively. Inspired by *CSR*, we propose *IOCSR index*, which uses a uniform index array to record the start and end offset addresses of vertices in the corresponding internal or external vector, as shown in Figure 6. With the *IOCSR index*, we can retrieve the adjacency list of any vertex rapidly no matter if it is in the internal or external vector, thereby avoiding the time-consuming vector scanning used in *EM_Dijk* [51].

Observe that although the loaded partitions have to be evicted before fully exploited due to the enlarging search front, in most cases, the partitions to be evicted would have been exploited by a considerable percentage if the main memory is large enough to hold a number of partitions. Based on this observation, we propose *Partition Shrinking*

Algorithm 4: Key Functions of I/O Management

```

1 // EV: External vector; IV: Internal vectors.
2 Function READPARTITION( $G_k, ifBi$ ):
3    $\langle ifevi, G_j \rangle \leftarrow LRU(G_k);$   $\triangleright$  LRU Strategy
4   if  $ifevi = true$  then
5     if  $RSize[G_j] < \alpha \cdot Size[G_j]$  then
6       foreach  $v \in IV[G_j]$  and  $used[v] = false$  do
7         if  $ifBi = false$  then
8           store  $N(v)$  to  $EV$ ;  $\triangleright$  PSS Strategy
9         else
10          if  $G_j \notin EV$  or  $used^r[v] = true$  then
11            store  $N(v)$  to  $EV$ ;  $\triangleright$  PSS Strategy
12       $Buf_G \leftarrow Buf_G \setminus G_j$ ;
13   READFROMDISK( $G_k$ );
14 Function BiREADPARTITION( $G_m, G_n$ ):
15   READPARTITION( $G_m, true$ );
16   if  $G_m \neq G_n$  then READPARTITION( $G_n, true$ );
17 Function STATUSUPDATE( $u_k, ifFwd$ ):
18   if  $ifFwd = true$  then
19      $n\_used[u_k] \leftarrow n\_used[u_k] + 1$ ;
20     if  $n\_used[u_k] = N$  then
21        $RSize[G_k] \leftarrow RSize[G_k] - deg(u_k)$ ;
22        $used[u_k] \leftarrow true$ ;
23       if  $RSize[G_k] = 0$  then  $Buf_G \leftarrow Buf_G \setminus G_k$ ;  $\triangleright$ 
24       Toss-Immediate Strategy
25   else
26     // Status update for reverse search. Simply replace
27      $n\_used, used$  by  $n\_used^r, used^r$  of Lines 19 to 22.

```

Strategy (PSS) to narrow down the size of underutilized partitions by removing the processed vertices and corresponding adjacency lists, thereby retaining only useful data to reduce the redundant I/O operations. Besides, for the partitions which have already been fully exploited, we adopt the **Toss-Immediate Strategy** to evict them immediately to make room for new partitions, and **LRU Strategy** is also utilized to manage the partitions.

PSS strategy aims to reduce redundant I/O operations by shrinking the size of underutilized partitions according to the usage of their vertices. There are two phases of PSS: **Update Phase** and **Eviction Phase**. In **Update Phase**, we update the number of times each vertex is used (i.e., n_used) after accessing it. If N criteria have accessed one vertex, we mark its status $used$ as $true$ and update the remaining edge size of the corresponding partition. In **Eviction Phase**, instead of evicting the whole partition, we only evict the used vertices and store other underutilized vertices in the external vector. It is worth noting that for **BMHP** and **BMHPS**, there may be multiple evictions of the same access partition when the forward search and reverse search intersect. Specifically, a partition G_i evicted during the forward search may be accessed again during the reverse search. For such a case, we only need to perform one additional reading operation to record the underutilized vertices in the external vector for future reference, as suggested by PSS.

The partition-based I/O management functions which adopt **LTP Strategy** are outlined in Algorithm 4. To be specific, the function **READPARTITION** is responsible for reading new partitions from disk and evicting old partitions (e.g., G_j) according to **LRU** strategy, if any. There are two approaches to eviction. One is to remove the partition

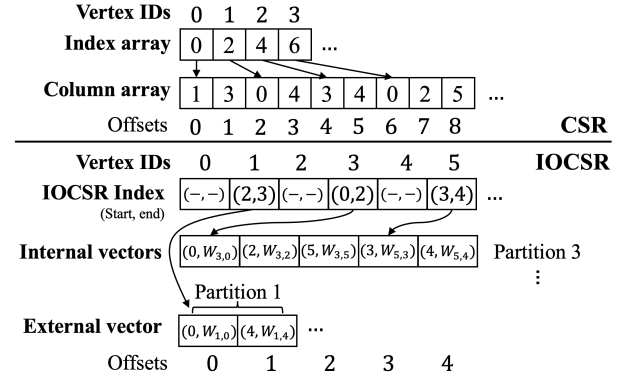


Fig. 6. CSR and IOCSR.

directly from the main memory, while the other one is to store the underutilized vertices into external STXXL vector EV according to the *Eviction Phase* of PSS (Lines 6 to 11). Observe that if the remaining size of G_j is too large, removing G_j from Buf_G directly might be more efficient than the second approach as no write operation is needed. Therefore, we introduce *eviction threshold* α to make tradeoffs between these two approaches. In particular, if the remaining size is larger than the original partition size times α , G_j is discarded directly; otherwise, the underutilized vertices of it are stored in the external vector. On the other hand, the function **STATUSUPDATE** is in charge of updating the status of the accessed vertex u_k and the remaining size of the corresponding partition G_k according to the *Update Phase* of PSS. Each time the status is updated, we check whether G_k has been fully exploited (i.e., the remaining size is zero), if so, we evict it immediately to make room for other partitions according to the toss-immediate strategy.

Example 8. Figure 6 depicts the CSR and IOCSR representation for graph in Example 2 when visit v_3 for OHP algorithm. In this case, partition 1 has been evicted from memory and pushed into an external vector, with only partition 3 residing in memory. According to PSS, only v_1 is pushed into external vector for future reference during the eviction of partition 1 while v_0 is evicted as it has been fully utilized.

6 EXPERIMENTAL EVALUATION

In this section, we evaluate the performance of the proposed methods and report the key findings. All the algorithms are implemented in C++ using STXXL library [16] and executed on a Dell R730 PowerEdge Rack Mount Server with two Xeon E5-2630 2.2GHz (each has 10 cores and 20 threads), 377G RAM, and a $12 \times 4TB$ HDDs (RAID50).

6.1 Experimental Settings

Datasets. We conduct experiments on eight large real-life graphs shown in Table 2. Among them, PopeElection2013 (PE)¹ and BostonBomb2013 (BB)¹ are weighted multiplex social networks obtained from Twitter, which have three criteria of relationship (retweeting, mentioning, and replying) among people during exceptional events. While Central USA (CT)³ and USA (US)³ are weighted road networks with

TABLE 2
Description of Real-world Datasets

Dataset	$ V $	$ E $	Type	Disk Size	Mem. Limit	T_{ave}
PE ¹	1,697,808	5,729,866	Social	0.17GB	0.13GB	0.05s
BB ¹	3,846,154	15,864,066	Social	0.57GB	0.50GB	0.12s
CT ³	14,081,816	33,866,826	Road	1.31GB	1.00GB	0.176s
US ³	23,947,347	57,708,624	Road	2.30GB	1.75GB	0.177s
OR ²	3,072,441	234,369,798	Social	8.20GB	5.00GB	0.025s
WI ²	13,591,759	669,181,586	Web	25.91GB	16.00GB	1.093s
TW ²	41,652,230	2,405,026,092	Social	86.26GB	48.00GB	6.232s
UK ²	104,288,749	6,587,610,160	Web	234.91GB	100.00GB	—

¹<https://manliodomenico.com/data.php> ²<http://konect.cc/>

³<http://www.diag.uniroma1.it/challenge9/download.shtml>

TABLE 3
Parameters Settings

Parameters	Values
Criteria number	2, 3, 4, 5
Aggregation strategy	PRI, AVE, WAVE
Partition size (MB)	0.25 , 0.5, 0.75, 1 , 1.5, 2, 2.5, 4, 15, 30
Memory size (GB)	0.13 , 0.5, 1.25, 1.5, 1.75, 2, 3, 4, 5, 6, 16, 48, 100

two criteria: distance and travel time. Orkut (OR)² and Twitter (TW)² are unweighted social networks while Wikipedia (WI)² and UK-2007-05 (UK)² are unweighted web graphs. Thus we generate the edge weight for their first criterion by formula $w_1(v_1, v_2) = 2 \cdot \max_{v \in V}(\deg(v)) / (\deg(v_1) + \deg(v_2))$, which is justified by the fact that in small-world graphs, the vertices with a higher degree usually have a higher impact on the whole graph, so it is easier to transmit information through these high-degree vertices. Besides, we follow existing work [7], [44], [55] to generate extra criteria for all datasets according to the correlated, independent, and anti-correlated distribution. In this way, all graphs are associated with five criteria where the first three criteria are correlated (PE and BB have three original criteria, while CT and US have two original criteria) whilst the fourth is anti-correlated, and the last one is independent.

Algorithms. We compare our proposed methods (*OHP*, *MHP*, *BMHP*, and *BMHPS*) with the multi-pass adaptation of classic SP methods: *MEMDijk*, *MDijk*, *MBiDijk*. Specifically, *MEMDijk* utilizes the state-of-the-art *EM_Dijk* [51] as underlying SP method. While *MDijk* and *MBiDijk* execute *Dijkstra's* algorithm [17] and *BiDijkstra's* algorithm [58] iteratively for each criterion, leveraging an improved I/O management strategy of *EM_Dijk*. In particular, we assign each partition the same hot pool as that in *EM_Dijk* (i.e., each hot pool consists of an internal multimap and an STXXL vector) to tackle drawback **D4** and manage these partitions by LRU policy. Moreover, the IOCSR vertex index mentioned in Section 5 is leveraged to retrieve the graph data in an external vector to avoid expensive linear search, thus solving **D1**. Table 3 reports the range values and default values (in boldface) of parameter settings. The default thread number of *BMHPS* is set to 40. We evaluate the performance of all algorithms on 100 random queries and report the average performance.

Partition Set. The default aggregation strategy is WAVE, the proportion of which is calculated according to the query distribution. The weight of criterion i is $\zeta_i = n_i / \sum_{i=1}^N(n_i)$, where n_i is the number of queries involving criterion i . The default partition sizes for PE, BB, CT, and US are 0.25MB,

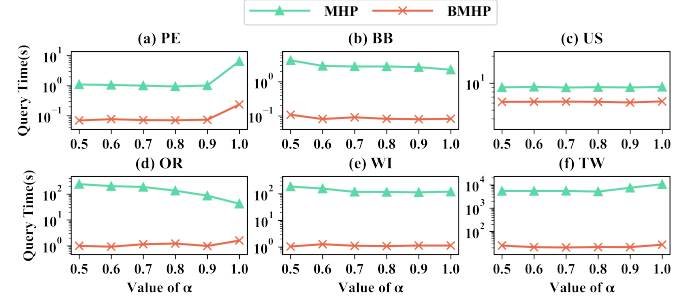


Fig. 7. Effect of α

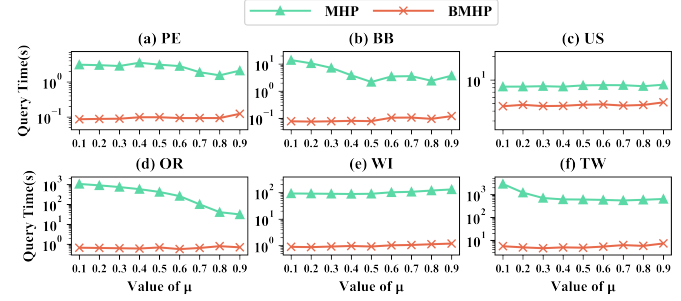


Fig. 8. Effect of μ

while OR is 1MB, WI is 4MB, TW is 15MB, and UK is 30MB.

Memory Set. It is worth noting that although the memory of our server is larger than the largest test graph, we limit the available memory size for all algorithms to be smaller than the disk size of $|E|$ to fulfill the memory setting of the semi-memory algorithm. As the graph sizes vary dramatically among the datasets, we set various default memory limits for them (see column 6 of Table 2).

Performance Measures. (1) *Query Time* in second; (2) I/O operation number (#I/O). Note that more than 100 queries are used for warm-up before running experiments, and we apply a warm cache for baselines (multi-pass methods) for each query so that the latter criterion may take advantage of it to reduce I/Os. If one query does not stop within 2 hours, we will not report its result. The I/O block size is 256 KB.

6.2 Experiment Results

Exp 1: Eviction Threshold α and Memory Ratio μ . As discussed in Section 4.3, unidirectional and bidirectional methods may have distinct search frontier sizes due to the different search spaces. Hence, we choose *MHP* and *BMHP* as the representatives of our unidirectional and bidirectional methods to evaluate the effect of α and μ in I/O management on part of datasets. We set μ as 0.8 by default for each dataset to seek the best α value and then search for the best μ value under the best α . As shown in Figures 7 and 8, *MHP* is very sensitive to different datasets while *BMHP* is more robust than them. Generally, a large α is conducive to the scenario when numerous large-size underutilized partitions must be evicted frequently since partition reloads are avoided, but it may burden the STXXL vector heavily. Whilst a small μ can load more partitions into the main memory but decreases the cache of external vector, which is not ideal for situations involving numerous evictions of under-used partitions. According to the experimental

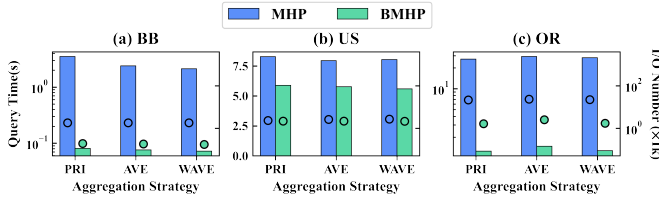


Fig. 9. Effect of Edge Aggregation Strategy (Bar: Time; Ball: I/O)

results, we set the parameters of the following experiments as follows (although they are not optimal, they still have satisfactory performance). For small-world networks, we set the α and μ of unidirectional methods as 0.9 and 0.8, while the α and μ of bidirectional methods as 0.9 and 0.5. For road networks, we set the α and μ of unidirectional methods as 0.9 and 0.4, while the α and μ of bidirectional methods as 0.6 and 0.3.

Exp 2: Effect of Edge Aggregation Strategy. Next, we choose *MHP* and *BMHP* to evaluate the proposed edge aggregation strategies for *deterministic graph clustering* method on BB, US, and OR. As shown in Figure 9, PRI performs worst in BB and US, while AVE performs badly in OR. This is because PRI only considers one criterion and therefore misses some useful information about other criteria, while AVE may involve “harmful” information as it treats all criteria equally. By contrast, WAVE associates more important criteria with higher weights, thus outperforming other competitors in most cases.

Exp 3: Effectiveness of Synchronization Strategy and I/O Management. We compare *OHP* and *MHP* with their LRU strategy-based variants (denoted as *OHP-LRU*, *MHP-LRU*) to demonstrate the effectiveness of search synchronization strategy and I/O management. As shown in Figure 10, *OHP* and *MHP* outperform *OHP-LRU* and *MHP-LRU* in all datasets. They achieve even more than 2 $\times$ speed up in most social networks (e.g., PE, BB, and OR) and about 40% improvement on road networks (CT and US). Relatively small gains are achieved in WI and TW because it is much easier to reach the “Tipping Point”. Hence, only a small proportion of the loaded partitions have been exploited before they are evicted, which greatly impairs the effectiveness of our I/O management. Besides, *MHP* always outperforms *OHP* due to the timely exploitation of in-memory information. As such, we use *MHP* as the default search synchronization strategy for our Stop& Share scheme in the following experiments.

Exp 4: Effectiveness of Stop& Share Optimizations. We next evaluate the proposed Optimizations by comparing *MHP* with *BMHP* and *BMHPS* on all datasets. The average time T_{ave} for one partition’s shortcut construction of *BMHPS* is listed in the last column of Table 2. Be aware that T_{ave} can be further reduced by using a smaller partition size or increasing the number of threads for parallel processing. Thus, *BMHPS* can cope with most dynamic environments with multi-thread parallelization. We do not report the results of *BMHPS* as it failed to finish the shortcut construction within two days, revealing the limitation of *BMHPS* on very large scale-free graphs as it has large degree and partition boundary number, which increases the shortcut number exponentially. In fact, only very lightweight indexes like

Sketch [19], [78] (pre-compute distance among tens of landmarks) could scale. Nevertheless, *BMHPS* is still an effective optimization for road networks and small-world networks with hundreds of millions of edges. The query processing results are shown in Figure 11, *BMHP* and *BMHPS* always outperforms *MHP* in all datasets, achieving a significant speed up (more than 10 $\times$) in small-world graphs. The first reason is that the meet-in-the-middle strategy can roughly halve the “radius” in search of one direction for small-world graphs (high skewness and average degree), thus it might bypass some high-degree vertices and significantly narrow down the search scope. Secondly, as mentioned in Section 3 that the search front grows exponentially with the expansion; hence it is easier for *OHP* and *MHP* to approach “Tipping Point” as they have to traverse more hops to reach the target vertex. On the other hand, *BMHPS* outperforms *BMHP* in road networks with a speed-up of 2 $\times$, and in small-world networks, it improves query efficiency by 10%–40%. In summary, both bidirectional technique and in-partition shortcut optimizations are conducive to improving performance and reducing I/O operations. In the following experiments, we use *BMHP* and *BMHPS* to represent our proposed methods as they perform best.

Exp 5: Comparison with Baselines. Next, we compare *MHP*, *BMHP*, and *BMHPS* with baselines on all datasets. As shown in Figure 12, *BMHP* and *BMHPS* always outperform all baselines in terms of both query time and I/O. For instance, *BMHPS* is about 4 $\times$ faster than *MBiDijk* in road networks while achieving about 20% – 80% improvement in small-world networks. Notice that the multi-pass version of *EM_Dijk* does not work well in most datasets and even cannot finish the test within the time limit on graph TW and UK. It is because although *EM_Dijk* also utilizes a hot pool to manage the loaded graph data efficiently, it still suffers from drawbacks mentioned in Section 5. It is worth noting that the good performance of *MDijk* and *MBiDijk* also further demonstrates the superiority of improved LRU-based partition management (which leverages *partition hot pool* and *IOCSR index*). Note that since *MDijk* outperforms *MEMDijk* in most cases and *MEMDijk* cannot finish the test within the time limit sometimes, we only use *MDijk* as a representative of the unidirectional search baseline in the following experiments. Besides, we choose *BMHP* to represent our methods in the following experiments to ensure the fairness of comparison.

Exp 6: Effect of the Number of Criteria. We further investigate the effect of the number of criteria (#Criteria) on US and OR by varying the number of criteria from 2 to 5, as shown in Figure 13-(a) and (b). In US, both the query time and I/O number of *MDijk* and *MBiDijk* are linear to the number of criteria, while *BMHP* has a much smaller linear slope in query time with a very small increase in #I/O. The former result is comprehensible because *MDijk* and *MBiDijk* simply iteratively execute the Dijkstra’s algorithm or Bi-Dijkstra’s algorithm for each criterion, thus the overall results are the summation of the costs of each criterion. Whilst *BMHP* can reduce the query time and #I/O by implementing the *Stop-and-Share Scheme*. Note that although warm cache has been applied for *MDijk* and *MBiDijk*, they do not get many cache hits because the graph data that the current criterion is demanding could have been evicted from the cache.

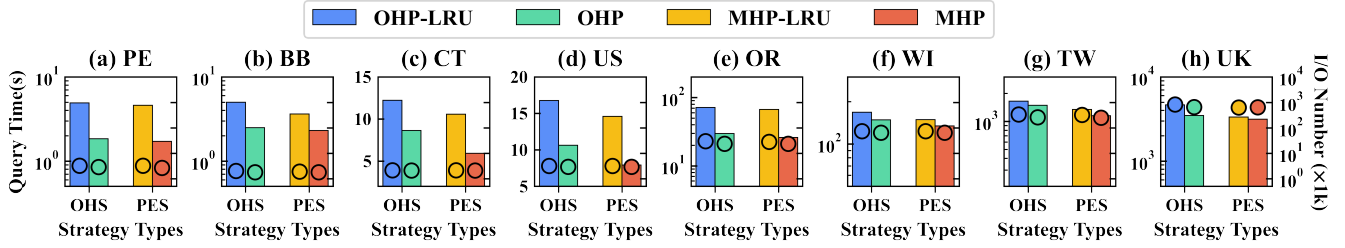


Fig. 10. Effectiveness of Synchronization Strategy and I/O Management (Bar: Time; Ball: I/O)

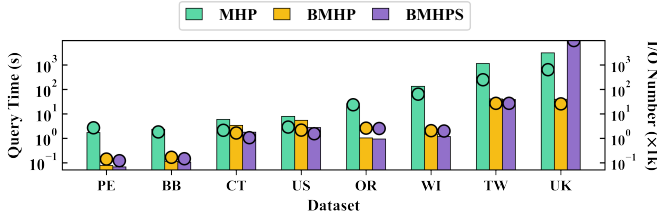


Fig. 11. Effectiveness of Optimizations (Bar: Time; Ball: I/O)

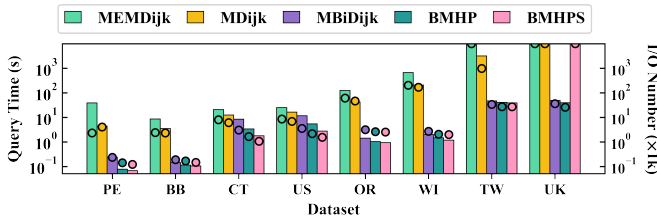


Fig. 12. Comparison with Baselines (Bar: Time; Ball: I/O)

By contrast, *BMHP* processes all criteria simultaneously to increase the cache hit. In OR, all algorithms drastically deteriorate the performance when we increase #Criteria from 3 to 4. This is because the 4th criterion is anti-correlated and thus more low-degree vertices will be expanded at first as their edges have lower weights, which results in a much larger search scope by considering the power-law degree distribution of small-world graphs. Overall, our proposed method is quite robust to any criteria type and can always outperform baselines regardless of how many criteria are involved.

Exp 7: Effect of Memory Size. We also evaluate the effect of memory size by varying the memory size in OR. Figure 13-(c) and (d) report the results. Observe that the query time and #I/O of *MDijk* and *MBiDijk* increase accordingly with the decrease of memory size in US while *BMHP* only has a noticeable rise when memory size decrease from 1.5GB to 1.25GB, which reveals that *BMHP* may have achieved almost optimal I/O operations when the memory size is larger than 1.5 GB. Note that we set the minimal memory size as 1.25 GB for US because the basic memory consumption of the *BMHP* algorithm (*i.e.*, the memory for recording the basic information of all vertices in *BMHP* algorithm) already exceeds 1.0 GB. Therefore, we cannot further cut down the memory size in order to compare *BMHP* with other competitors. In OR, the query time and #I/O of *MDijk* exploded exponentially with the decrease of memory size and it cannot complete the test when the memory size is 3.0 GB. This is because too small memory size makes it much

TABLE 4
Query Time Results on Flash Memory

Methods	BB		US		OR	
	HDD (s)	SSD (s)	HDD (s)	SSD (s)	HDD (s)	SSD (s)
MEMDijk	8.133	4.113	35.217	29.428	363.423	60.982
MDijk	3.355	1.779	16.258	12.452	56.515	36.510
MBiDijk	0.184	0.080	9.548	6.379	1.287	0.949
MHP	2.308	1.347	7.941	7.452	25.950	23.393
BMHP	0.074	0.058	5.399	4.428	1.029	0.855
BMHPS	0.055	0.039	2.771	2.074	0.930	0.741

easier to reach the “Tipping Point” thus involving a large number of underutilized partition evictions. By contrast, *BMHP* can still remain high efficiency and achieve the best performance of the proposed methods, which further demonstrates the superiority of *PSS* strategy.

Exp 8: Effect of Partition Size. Next, we investigate the effect of partition size by varying partition size from 0.25 MB to 1.0 MB in US and from 1.0 MB to 2.5 MB in OR. The results are shown in Figure 13-(e) and (f). Observe that the query time of all algorithms undergoes a decreasing trend in US and an increasing trend in OR with the increase of partition size. Meanwhile, there is no fluctuation for *BMHP* in #I/O of US while an increasing trend in #I/O of OR as can be observed in Figure 13-(f). The reason may lie in that most loaded partitions in US can be fully exploited while many underutilized partitions have to be evicted in OR. Hence, a larger partition size does not increase too many I/Os in US but decreases the average disk seek time as fewer partition files are involved. By contrast, a larger partition size causes more I/Os and results in worse performance in terms of query time in OR. This experiment implies that a smaller partition size is conducive to large small-world graphs but may not benefit road networks in terms of query time.

Exp 9: Preliminary Results on Flash Memory. Lastly, we also conduct preliminary experiments of the proposed methods on solid state disks (SSDs) to evaluate the effect of external storage medium. The tests are performed on a macOS system with one 8-core Intel Core i9-9980HK CPU@2.40GHz, 32GB RAM, and 2TB SSDs. Note that we only report the query time as the #I/O is the same as Exp 3. As shown in Table 4, all methods achieve significant improvements on SSD, especially for those involving massive I/Os. For example, *MEMDijk* can achieve more than 5× faster performance in SSDs. Besides, the fastest method is still *BMHPS*, which on average can answer a three-criteria MCSPs query in 2.1 seconds on US.

7 RELATED WORK

Most germane to this research is shortest path computation methods and SP query processing on multi-criteria graphs.

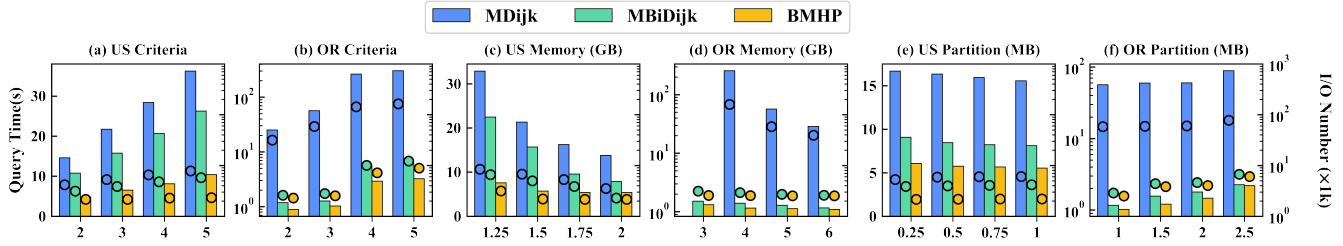


Fig. 13. Effect of the Number of Criteria, Memory Size, and Partition Size (Bar: Time; Ball: I/O)

7.1 Shortest Path Computation

7.1.1 In-memory Shortest Path Algorithms

In-memory SP algorithms first load the entire graph into memory and then compute the shortest paths over the memory-resident network. The *index-free* methods [17], [24], [29], [58], [85] search directly in the network and require none or very little preprocessing so they can cope with the dynamic scenario naturally. The *index-based* method precomputes different auxiliary information to speed up query answering [3], [13], [22], [38], [42], [43], [54], [60], [64]. In general, the larger the index, the shorter the query time, and the longer time to construct and maintain, so they are not preferable as tested in [87]. Nevertheless, none of them consider the I/O efficiency, especially over a multi-criteria graph: the *index-free* methods require the entire graph to fit in memory [31], while the *index-based* methods further need much larger memory for index construction and maintenance [31], [86], so they cannot solve our problem.

7.1.2 I/O-Efficient Shortest Path Computations

As graphs grow rapidly in size and can hardly fit in memory, external memory path algorithms [1], [2], [8], [11], [36], [50], [51], [53], [56], [90] are proposed to reduce the I/O cost. Specifically, *EM_Dijk* [51] is an I/O-efficient semi-external algorithm that exploits the data locality by adopting deterministic graph clustering. Besides, it adopts hot pooling techniques to reduce random disk access. To our best knowledge, it is still the state-of-the-art index-free I/O efficient SSSP algorithm for weighted graphs with the I/O complexity of $O(\sqrt{|V||E|K/B} + MST(|V|, |E|))$ [51], where B is the block size, K is largest weight and $MST(|V|, |E|)$ is the cost of computing the minimum spanning tree. In contrast, *VC-Index* [11] is an I/O-efficient index-based algorithm that precomputes a number of reduced graphs. It outperforms *EM_Dijk* as it only requires sequential read, and *HoD-index* [90] further improves its efficiency. However, these algorithms are designed for computing SSSP queries, which are wasteful for computing pairwise *MCSPs* queries studied in the paper. Although Fu *et al.* proposed *IS-Label* [21] to compute point-to-point shortest path queries, it is not designed for multi-criteria graphs. All these I/O-efficient SP indexes may suffer from huge label sizes [31]. Besides, they are time-consuming to maintain since they have to be reconstructed when the graph evolves.

7.1.3 Out-of-Core Graph Systems

Vertex-centric out-of-core graph systems [14], [49], [57], [81], [91] can process graphs with billions or trillions of edges by utilizing external memory on a single machine.

However, their objectives primarily involve efficient data representation and system design to support general graph algorithms such as *PageRank*, *Weakly Connected Components (WCC)*, *Breadth-First Search (BFS)*, *SSSP*, and others, rather than SP algorithm design. The algorithms they apply have to involve all the vertices simultaneously to maximize the parallelism. Therefore, they utilize *Bellman-Ford* [5] for *SSSP*, which has a higher complexity $O(|V| \cdot |E|)$ and rarely used in real-life, instead of *Dijkstra*. In fact, it is caused by the fact that they are essentially adaptations of *MapReduce* [15] to the graph data model. As validated over the past decade, only a small portion of problem and algorithm could benefit from such a computation paradigm, and *SP* with a fixed destination is not among them. Therefore, to solve the *MC-SPs* problem, they still have to iteratively perform *Dijkstra's* algorithm for each criterion, thus missing the opportunity to share the computation among different criteria.

7.1.4 Concurrent SP Query Processing

Concurrent shortest path query processing [41], [83], [84] aims to simultaneously execute multiple shortest path queries to improve high throughput query efficiency. A typical approach [41] is to decompose the SP queries according to their intimacy and then leverage batch algorithms (*e.g.*, caching techniques [72], [73] or A^*1N [83]) to execute the clustered queries in batch. These methods cannot be applied to *MCSPs* problem as they aim to deal with multiple queries of different OD pairs. Besides, some graph systems [9], [62] also deal with concurrent SP query processing by leveraging parallel SP algorithms such as Δ -stepping algorithm [52]. They mainly focus on system-level optimization such as dynamic query scheduler [62]. However, all these methods only deal with graphs with a single criterion.

7.2 Queries on Multi-Criteria Graphs

Depending on if the criteria have constraints or not, the multi-criteria graph paths fall into the following two types:

7.2.1 Skyline Path

Like the skyline queries [63], it returns a set of paths that cannot dominate each other from all criteria [25], [35], [61]. Apparently, the shortest path of one criterion must be a skyline path over other possible paths, so we can utilize the skyline path algorithm for our problem. Nonetheless, its complexity is $O(w_{max}^{N-1}|V|(|V|\log(|V|) + |E|))$ where w_{max} is the largest weight and N is criteria number [28], which is much larger than running *Dijkstra's* algorithm iteratively.

7.2.2 Constrained Shortest Path (CSP)

CSP is more applicable in real-life as it only returns one shortest path that satisfies constraints on other criteria. As it is a special case of the skyline path, it is still very time-consuming to search directly [10], [27], [28], [48], [65], [88] for CSP. As CSP is known as NP-Hard [30], so approximate algorithms are more appealing [28], [30], [32], [47], [75]. To further improve the query performance, indexes are also extended to CSP [44], [46], [69], [77]. Nevertheless, they still suffer from the high complexity inherited from the skyline path and cannot solve our problem efficiently.

8 CONCLUSION

In this paper, we formally investigate the problem of processing multi-criteria shortest paths queries on large disk-resident graphs for the first time. To efficiently process these queries, we present a generic and extensible STOP & SHARE scheme and four index-free semi-memory algorithms (OHP, MHP, BMHP, and BMHPS) to share I/O computations for multiple criteria. The corresponding partition-based I/O management optimizations are also investigated. Extensive experiments on several large real-world datasets validate the effectiveness and efficiency of our methods.

REFERENCES

- [1] D. Ajwani, R. Dementiev, and U. Meyer. A computational study of external-memory bfs algorithms. In *The Seventeenth Annual ACM-SIAM Symposium on Discrete Algorithm (SODA'06)*, Miami, Florida, 22-26 January 2006. ACM, 2006.
- [2] D. Ajwani, U. Meyer, and V. Osipov. Improved external memory bfs implementations. In *2007 Proceedings of the Ninth Workshop on Algorithm Engineering and Experiments (ALENEX)*, pages 3–12. SIAM, 2007.
- [3] T. Akiba, Y. Iwata, and Y. Yoshida. Fast exact shortest-path distance queries on large networks by pruned landmark labeling. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, pages 349–360, 2013.
- [4] A. Batzill. Optimal route planning on mobile systems. *Masterarbeit, Hochschule*, 2016.
- [5] R. Bellman. On a routing problem. *Quarterly of applied mathematics*, 16(1):87–90, 1958.
- [6] T. Blsius, J. Eube, T. Feldtkeller, T. Friedrich, M. S. Krejca, J. G. Lagodzinski, R. Rothenberger, J. Severin, F. Sommer, and J. Trautmann. Memory-restricted routing with tiled map data. In *2018 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pages 3347–3354. IEEE, 2018.
- [7] S. Borzsony, D. Kossmann, and K. Stocker. The skyline operator. In *Proceedings 17th international conference on data engineering*, pages 421–430. IEEE, 2001.
- [8] A. L. Buchsbaum, M. H. Goldwasser, S. Venkatasubramanian, and J. R. Westbrook. On external memory graph traversal. In *SODA*, pages 859–860, 2000.
- [9] R. Chen and H. Chen. Wukong: A distributed framework for fast and concurrent graph querying. *ACM SIGOPS Operating Systems Review*, 55(1):77–83, 2021.
- [10] X. Chen, X. Cao, Y. Zeng, Y. Fang, S. Wang, X. Lin, and L. Feng. Constrained path search with submodular function maximization. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*, pages 325–337. IEEE, 2022.
- [11] J. Cheng, Y. Ke, S. Chu, and C. Cheng. Efficient processing of distance queries in large graphs: a vertex cover approach. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, pages 457–468, 2012.
- [12] E. G. Coffman and P. J. Denning. *Operating systems theory*, volume 973. prentice-Hall Englewood Cliffs, NJ, 1973.
- [13] E. Cohen, E. Halperin, H. Kaplan, and U. Zwick. Reachability and distance queries via 2-hop labels. *SIAM Journal on Computing*, 32(5):1338–1355, 2003.
- [14] D. M. Da Zheng, R. Burns, J. Vogelstein, C. E. Priebe, and A. S. Szalay. Flashgraph: Processing billion-node graphs on an array of commodity ssds. In *13th USENIX Conference on File and Storage Technologies (FAST 15)*, pages 45–58, 2015.
- [15] J. Dean and S. Ghemawat. Mapreduce: Simplified data processing on large clusters. In *Proceedings of the 6th Conference on Symposium on Operating Systems Design & Implementation - Volume 6, OSDI'04*, page 10, USA, 2004. USENIX Association.
- [16] R. Dementiev, L. Kettner, and P. Sanders. Stxxl: standard template library for xxl data sets. *Software: Practice and Experience*, 38(6):589–637, 2008.
- [17] E. W. Dijkstra et al. A note on two problems in connexion with graphs. *Numerische mathematik*, 1(1):269–271, 1959.
- [18] W. Fan, Y. Li, M. Liu, and C. Lu. A hierarchical contraction scheme for querying big graphs. In *Proceedings of the 2022 International Conference on Management of Data*, pages 1726–1740, 2022.
- [19] M. Farhan, Q. Wang, and H. Koehler. Batchhl: Answering distance queries on batch-dynamic networks at scale. In *Proceedings of the 2022 International Conference on Management of Data*, pages 2020–2033, 2022.
- [20] L. C. Freeman. Visualizing social networks. *Journal of social structure*, 1(1):4, 2000.
- [21] A. W.-C. Fu, H. Wu, J. Cheng, and R. C.-W. Wong. Is-label: an independent-set based labeling scheme for point-to-point distance querying. *Proceedings of the VLDB Endowment*, 6(6):457–468, 2013.
- [22] R. Geisberger, P. Sanders, D. Schultes, and D. Delling. Contraction hierarchies: Faster and simpler hierarchical routing in road networks. In *International workshop on experimental and efficient algorithms*, pages 319–333. Springer, 2008.
- [23] S. Ghariblou, M. Salehi, M. Magnani, and M. Jalili. Shortest paths in multiplex networks. *Scientific reports*, 7(1):1–8, 2017.
- [24] A. V. Goldberg and C. Harrelson. Computing the shortest path: A search meets graph theory. In *SODA*, volume 5, pages 156–165. Citeseer, 2005.
- [25] Q. Gong, H. Cao, and P. Nagarkar. Skyline queries constrained by multi-cost transportation networks. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, pages 926–937. IEEE, 2019.
- [26] A. Gorbenco and V. Popov. On the longest common subsequence problem. *Applied Mathematical Sciences*, 6(116):5781–5787, 2012.
- [27] G. Y. Handler and I. Zang. A dual algorithm for the constrained shortest path problem. *Networks*, 10(4):293–309, 1980.
- [28] P. Hansen. Bicriterion path problems. In *Multiple criteria decision making theory and application*, pages 109–127. Springer, 1980.
- [29] P. E. Hart, N. J. Nilsson, and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.
- [30] R. Hassin. Approximation schemes for the restricted shortest path problem. *Mathematics of Operations research*, 17(1):36–42, 1992.
- [31] M. Jiang, A. W.-C. Fu, R. C.-W. Wong, and Y. Xu. Hop doubling label indexing for point-to-point distance querying on scale-free networks. *arXiv preprint arXiv:1403.0779*, 2014.
- [32] A. Juttner, B. Szviatevski, I. Mécs, and Z. Rajkó. Lagrange relaxation based method for the qos routing problem. In *Proceedings IEEE INFOCOM 2001. Conference on Computer Communications. Twentieth Annual Joint Conference of the IEEE Computer and Communications Society (Cat. No. 01CH37213)*, volume 2, pages 859–868. IEEE, 2001.
- [33] R. Kanawati. Multiplex network mining: A brief survey. *IEEE Intell. Informatics Bull.*, 16(1):24–27, 2015.
- [34] T. Korkmaz and M. Krunz. Multi-constrained optimal path selection. In *Proceedings IEEE INFOCOM 2001. Conference on Computer Communications. Twentieth Annual Joint Conference of the IEEE Computer and Communications Society (Cat. No. 01CH37213)*, volume 2, pages 834–843. IEEE, 2001.
- [35] H.-P. Kriegel, M. Renz, and M. Schubert. Route skyline queries: A multi-preference path planning approach. In *2010 IEEE 26th International Conference on Data Engineering (ICDE 2010)*, pages 261–272. IEEE, 2010.
- [36] V. Kumar and E. J. Schwabe. Improved algorithms and data structures for solving graph problems in external memory. In *Proceedings of SPDP'96: 8th IEEE Symposium on Parallel and Distributed Processing*, pages 169–176. IEEE, 1996.
- [37] J. Lerner, D. Wagner, and K. Zweig. *Algorithmics of large and complex networks: design, analysis, and simulation*, volume 5515. Springer, 2009.

- [38] L. Li, W. Hua, X. Du, and X. Zhou. Minimal on-road time route scheduling on time-dependent graphs. *Proceedings of the VLDB Endowment*, 10(11):1274–1285, 2017.
- [39] L. Li, S. Wang, and X. Zhou. Time-dependent hop labeling on road network. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, pages 902–913. IEEE, 2019.
- [40] L. Li, S. Wang, and X. Zhou. Fastest path query answering using time-dependent hop-labeling in road network. *IEEE Transactions on Knowledge and Data Engineering*, 2020.
- [41] L. Li, M. Zhang, W. Hua, and X. Zhou. Fast query decomposition for batch shortest path processing in road networks. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*, pages 1189–1200. IEEE, 2020.
- [42] L. Li, K. Zheng, S. Wang, W. Hua, and X. Zhou. Go slow to go fast: minimal on-road time route scheduling with parking facilities using historical trajectory. *The VLDB Journal*, 27(3):321–345, 2018.
- [43] Y. Li, L. H. U, M. L. Yiu, and N. M. Kou. An experimental study on hub labeling based shortest path algorithms. *Proceedings of the VLDB Endowment*, 11(4):445–457, 2017.
- [44] Z. Liu, L. Li, M. Zhang, W. Hua, P. Chao, and X. Zhou. Efficient constrained shortest path query answering with forest hop labeling. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, pages 1763–1774. IEEE, 2021.
- [45] Z. Liu, L. Li, M. Zhang, W. Hua, and X. Zhou. Fhl-cube: Multi-constraint shortest path querying with flexible combination of constraints. *Proceedings of the VLDB Endowment*, 15, 2022.
- [46] Z. Liu, L. Li, M. Zhang, W. Hua, and X. Zhou. Multi-constraint shortest path using forest hop labeling. *The VLDB Journal*, pages 1–27, 2022.
- [47] D. H. Lorenz and D. Raz. A simple efficient approximation scheme for the restricted shortest path problem. *Operations Research Letters*, 28(5):213–219, 2001.
- [48] L. Lozano and A. L. Medaglia. On an exact method for the constrained shortest path problem. *Computers & Operations Research*, 40(1):378–384, 2013.
- [49] S. Maass, C. Min, S. Kashyap, W. Kang, M. Kumar, and T. Kim. Mosaic: Processing a trillion-edge graph on a single machine. In *Proceedings of the Twelfth European Conference on Computer Systems*, pages 527–543, 2017.
- [50] K. Mehlhorn and U. Meyer. External-memory breadth-first search with sublinear i/o. In *European Symposium on Algorithms*, pages 723–735. Springer, 2002.
- [51] U. Meyer and V. Osipov. Design and implementation of a practical i/o-efficient shortest paths algorithm. In *2009 Proceedings of the Eleventh Workshop on Algorithm Engineering and Experiments (ALENEX)*, pages 85–96. SIAM, 2009.
- [52] U. Meyer and P. Sanders. δ -stepping: a parallelizable shortest path algorithm. *Journal of Algorithms*, 49(1):114–152, 2003.
- [53] U. Meyer and N. Zeh. I/o-efficient undirected shortest paths with unbounded edge lengths. In *European Symposium on Algorithms*, pages 540–551. Springer, 2006.
- [54] R. H. Möhring, H. Schilling, B. Schütz, D. Wagner, and T. Willhalm. Partitioning graphs to speedup dijkstra's algorithm. *Journal of Experimental Algorithmics (JEA)*, 11:2–8, 2007.
- [55] K. Mouratidis, Y. Lin, and M. L. Yiu. Preference queries in large multi-cost transportation networks. In *2010 IEEE 26th International Conference on Data Engineering (ICDE 2010)*, pages 533–544. IEEE, 2010.
- [56] K. Munagala and A. Ranade. I/o-complexity of graph algorithms. In *SODA*, volume 99, pages 687–694. Citeseer, 1999.
- [57] D. Nguyen, A. Lenharth, and K. Pingali. A lightweight infrastructure for graph analytics. In *Proceedings of the twenty-fourth ACM symposium on operating systems principles*, pages 456–471, 2013.
- [58] T. A. J. Nicholson. Finding the shortest route between two points in a network. *The computer journal*, 9(3):275–280, 1966.
- [59] E. Omodei, M. De Domenico, and A. Arenas. Characterizing interactions in online social networks during exceptional events. *Frontiers in Physics*, 3:59, 2015.
- [60] D. Ouyang, L. Qin, L. Chang, X. Lin, Y. Zhang, and Q. Zhu. When hierarchy meets 2-hop-labeling: Efficient shortest distance queries on road networks. In *Proceedings of the 2018 International Conference on Management of Data*, pages 709–724, 2018.
- [61] D. Ouyang, L. Yuan, F. Zhang, L. Qin, and X. Lin. Towards efficient path skyline computation in bicriteria networks. In *International Conference on Database Systems for Advanced Applications*, pages 239–254. Springer, 2018.
- [62] P. Pan and C. Li. Congra: Towards efficient processing of concurrent graph queries on shared-memory machines. In *2017 IEEE International Conference on Computer Design (ICCD)*, pages 217–224. IEEE, 2017.
- [63] D. Papadias, Y. Tao, G. Fu, and B. Seeger. An optimal and progressive algorithm for skyline queries. In *Proceedings of the 2003 ACM SIGMOD international conference on Management of data*, pages 467–478, 2003.
- [64] H. Samet, J. Sankaranarayanan, and H. Alborzi. Scalable network distance browsing in spatial databases. In *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*, pages 43–54, 2008.
- [65] A. Sedeño-Noda and S. Alonso-Rodríguez. An enhanced k-sp algorithm with pruning strategies to solve the constrained shortest path problem. *Applied Mathematics and Computation*, 265:602–618, 2015.
- [66] J. Shang, Y. Zheng, W. Tong, E. Chang, and Y. Yu. Inferring gas consumption and pollution emission of vehicles throughout a city. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 1027–1036, 2014.
- [67] A. Silberschatz, H. F. Korth, S. Sudarshan, et al. *Database system concepts*, volume 5. McGraw-Hill New York, 2002.
- [68] A. Solé-Ribalta, M. De Domenico, S. Gómez, and A. Arenas. Centrality rankings in multiplex networks. In *Proceedings of the 2014 ACM conference on Web science*, pages 149–155, 2014.
- [69] S. Storandt. Route planning for bicycles—exact constrained shortest paths made practical via contraction hierarchy. In *Twenty-second international conference on automated planning and scheduling*, 2012.
- [70] A. Strehl, J. Ghosh, and R. Mooney. Impact of similarity measures on web-page clustering. In *Workshop on artificial intelligence for web search (AAAI 2000)*, volume 58, page 64, 2000.
- [71] P. Sun, Y. Wen, T. N. B. Duong, and X. Xiao. Graphmp: I/o-efficient big graph analytics on a single commodity machine. *IEEE Transactions on Big Data*, 6(4):816–829, 2019.
- [72] J. R. Thomsen, M. L. Yiu, and C. S. Jensen. Effective caching of shortest paths for location-based services. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, pages 313–324, 2012.
- [73] J. R. Thomsen, M. L. Yiu, and C. S. Jensen. Concise caching of driving instructions. In *Proceedings of the 22nd ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, pages 23–32, 2014.
- [74] W. F. Tinney and J. W. Walker. Direct solutions of sparse network equations by optimally ordered triangular factorization. *Proceedings of the IEEE*, 55(11):1801–1809, 1967.
- [75] G. Tsaggouris and C. Zaroliagis. Multiobjective optimization: Improved fptas for shortest paths and non-linear objectives with applications. *Theory of Computing Systems*, 45(1):162–186, 2009.
- [76] P. Van Mieghem and F. A. Kuipers. On the complexity of qos routing. *Computer communications*, 26(4):376–387, 2003.
- [77] S. Wang, X. Xiao, Y. Yang, and W. Lin. Effective indexing for approximate constrained shortest path queries on large road networks. *Proceedings of the VLDB Endowment*, 10(2):61–72, 2016.
- [78] Y. Wang, Q. Wang, H. Koehler, and Y. Lin. Query-by-sketch: Scaling shortest path graph queries on very large networks. In *Proceedings of the 2021 International Conference on Management of Data*, pages 1946–1958, 2021.
- [79] Z. Wang, Y. Gu, Y. Bao, G. Yu, and J. X. Yu. Hybrid pulling/pushing for i/o-efficient distributed and iterative graph computing. In *Proceedings of the 2016 International Conference on Management of Data*, pages 479–494, 2016.
- [80] B. Wheatman and H. Xu. Packed compressed sparse row: A dynamic graph representation. In *2018 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–7. IEEE, 2018.
- [81] M. Wu, F. Yang, J. Xue, W. Xiao, Y. Miao, L. Wei, H. Lin, Y. Dai, and L. Zhou. Gram: Scaling graph computation to the trillions. In *Proceedings of the Sixth ACM Symposium on Cloud Computing*, pages 408–421, 2015.
- [82] M. Zhang, L. Li, W. Hua, R. Mao, P. Chao, and X. Zhou. Dynamic hub labeling for road networks. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, pages 336–347. IEEE, 2021.
- [83] M. Zhang, L. Li, W. Hua, and X. Zhou. Batch processing of shortest path queries in road networks. In *Databases Theory and Applications: 30th Australasian Database Conference, ADC 2019, Sydney, NSW, Australia, January 29–February 1, 2019, Proceedings 30*, pages 3–16. Springer, 2019.

- [84] M. Zhang, L. Li, W. Hua, and X. Zhou. Efficient batch processing of shortest path queries in road networks. In *2019 20th IEEE International Conference on Mobile Data Management (MDM)*, pages 100–105. IEEE, 2019.
- [85] M. Zhang, L. Li, W. Hua, and X. Zhou. Stream processing of shortest path query in dynamic road networks. *IEEE Transactions on Knowledge and Data Engineering*, 2020.
- [86] M. Zhang, L. Li, W. Hua, and X. Zhou. Efficient 2-hop labeling maintenance in dynamic small-world networks. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, pages 133–144. IEEE, 2021.
- [87] M. Zhang, L. Li, and X. Zhou. An experimental evaluation and guideline for path finding in weighted dynamic network. *Proceedings of the VLDB Endowment*, 14(11):2127–2140, 2021.
- [88] X. Zhang, Z. Liu, M. Zhang, and L. Li. An experimental study on exact multi-constraint shortest path finding. In *Australasian Database Conference*, pages 166–179. Springer, 2021.
- [89] Z. Zhang, J. X. Yu, L. Qin, and Z. Shang. Divide & conquer: I/o efficient depth-first search. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, pages 445–458, 2015.
- [90] A. D. Zhu, X. Xiao, S. Wang, and W. Lin. Efficient single-source shortest path and distance queries on large graphs. In *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 998–1006, 2013.
- [91] X. Zhu, W. Han, and W. Chen. Gridgraph: Large-scale graph processing on a single machine using 2-level hierarchical partitioning. In *USENIX Annual Technical Conference*, pages 375–386, 2015.



Xiaofang Zhou received his Bachelor and Master in Computer Science degrees from Nanjing University in 1984 and 1987 respectively, and his PhD in Computer Science degree from the University of Queensland in 1994. He is Otto Poon Professor of Engineering and Chair Professor of Computer Science and Engineering at the Hong Kong University of Science and Technology. His research interests include spatial and multimedia databases, high performance query processing, web information systems, data mining, and data quality management. He is a Fellow of IEEE.



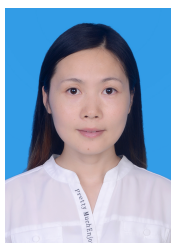
Xinjie Zhou is currently working toward the PhD degree at the Hong Kong University of Science and Technology under the supervision of Prof. Xiaofang Zhou and Dr. Lei Li. He received his Bachelor's degree from Xi'an Jiaotong University in 2018 and his Master's degree from Shanghai Jiao Tong University in 2021. His current research interests include scalable graph database management and shortest path algorithms.



Kai Huang is an Assistant Professor at the Department of Computer Science and Engineering, MUST. He received his PhD degree in School of Computer Science from Fudan University in 2020, and BEng degree in Software Engineering from East China Normal University in 2014. His research interests include graph database and privacy-aware data management.



Lei Li is an Assistant Professor at the Hong Kong University of Science and Technology (Guangzhou). He obtained his Ph.D. in 2018 from the University of Queensland under Prof. Xiaofang Zhou's supervision. He obtained his Bachelor's and Master's degrees from Harbin Institute of Technology in 2012 and 2014, respectively. His research interests include spatial and temporal data, graph, and distributed databases.



Mengxuan Zhang is currently a Postdoc Research Associate in the University of North Texas. She obtained her Ph.D. in 2021 from the University of Queensland under Prof. Xiaofang Zhou's supervision. She received her Bachelor of Engineering (Automation) in 2014 from Wuhan University of Science and Technology and her Master of Engineering (Control Theory and Control Engineering) in 2017 from Dalian University of Technology. Her research interests include spatial databases and dynamic route

planning.